

Acoustic Occupation

Salience, Recovery, and the Temporal Organization of Film Sound

Flyxion
Independent Researcher

September 2026

Working manuscript

Abstract

Complaints about the sound of contemporary films are usually framed as complaints about loudness. This monograph argues that the variable doing the damage is not level but occupation: the proportion and continuity of time during which a soundtrack stays well above the level at which its dialogue is intelligible, together with the scarcity of intervals in which the listener is allowed to recover. The argument is developed through a single worked case, the 2026 action feature *Drive Through Fire*, reproduced on a small bass instrument amplifier from a computer headphone output, where the listener repeatedly reduced the volume to zero for one to two minutes at a time during gunfire and fight sequences. Measured against its own subtitled dialogue with BS.1770 loudness, the film spends almost a third of its audible running time at least 10 dB above dialogue level, on a plateau rather than in extreme peaks, and contains a twenty-minute stretch in which the soundtrack returns to dialogue level for only half a minute; standard loudness figures for the same film show none of this. The monograph presents the measurement pipeline built to analyse that soundtrack, documents in detail the ways its first versions failed, introduces a dialogue-referenced definition of occupation that avoids the circularity of percentile thresholds, and specifies a response measure (logged volume throttling) and an intervention (a dialogue-forward downmix) that together turn a single listening report into a falsifiable research programme. Claims are graded throughout as established, measured in this study, hypothesised, or rejected.

Preface

This study began with a practical failure of listening. A feature film was playing through a small bass practice amplifier connected to the headphone socket of a computer, and for long stretches the only workable response to its soundtrack was to turn the volume all the way down and wait for the scene to end. Dialogue, when it returned, was too quiet to follow at the setting that had made the preceding gunfire tolerable. The volume went up for speech and down again for action, over and over, and some action sequences outlasted any patience for riding the control.

That experience is the only direct observation of listener behaviour in the monograph, and it is treated accordingly. It is reported once, in Chapter 7, as the report of a single listener on a single viewing through an unmeasured playback chain. Everything else in the book either measures the soundtrack itself, describes the instruments used to do so, or specifies what would have to be collected for the report to become evidence rather than motivation.

The book is organised as a sequence of instruments and their limits. Part I states the problem and the conceptual framework. Part II describes the analysis pipeline and, at some length, the ways its earlier versions produced confident nonsense: a swell detector that was an inverted transient detector, harshness scores that peaked in digital silence, and summary statistics whose values were fixed in advance by the thresholds that produced them. These failures are kept in the text because they are instructive and because a reader deserves to know how the current numbers were reached. Part III applies the corrected instruments to the case. Part IV specifies a response measure and an intervention, both of which were first sketched as speculative tools in July 2025 and are here developed into a research design. Part V grades what has and has not been shown.

Numerical results that depend on the dialogue-referenced analysis of Chapter 3 are generated by `code/occupation.py` and `code/occupation-extras.py` and inserted automatically. In this build they have been generated from the analysis tables of the film.

The code, the figures, and the manuscript source are released into the public domain.

Contents

Abstract	ii
Preface	iii
I The Problem	1
1 Occupation, Not Loudness	2
1.1 A familiar complaint, stated precisely	2
1.2 Peak intensity and occupation	2
1.3 Why a single case	3
1.4 Grading claims	3
1.5 Plan of the book	3
2 Background: Standards, Practice, and Perception	4
2.1 Measuring loudness	4
2.2 Dialogue as reference	5
2.3 Cinema calibration and the home	5
2.4 Film sound practice and theory	5
2.5 Psychoacoustics	6
2.6 Noise, annoyance, and control	6
2.7 Sound, affect, and the soundscape	6
2.8 The gap	7
3 A Framework for Occupation	8
3.1 Time base and level	8
3.2 The dialogue reference	8
3.3 Exceedance and occupation	9
3.4 Recovery	10
3.5 Salience channels	10
3.6 A listener model	11
3.7 Hypotheses	11

II	Instruments	13
4	The Analysis Pipeline	14
4.1	Design constraints	14
4.2	Analysis stems	14
4.3	Frame features	15
4.4	Audibility weighting	15
4.5	Detectors	15
4.6	Audition clips	16
4.7	Loudness	16
4.8	Subtitles	16
4.9	Parameters	17
5	How the Detectors Failed	18
5.1	The swell detector that detected transients	18
5.2	Harshness in silence	19
5.3	Statistics fixed by their thresholds	20
5.4	Interpretive overreach	21
5.5	Rules adopted	21
6	The Reproduction Chain	22
6.1	The chain in the case	22
6.2	Four transformations	22
6.2.1	Channel reduction	22
6.2.2	Band limitation	23
6.2.3	Level-dependent distortion	23
6.2.4	Coupling of volume and distortion	23
6.3	How the chain enlarges exceedance	23
6.4	Measuring the chain	24
6.5	Simulating the chain	24
III	The Case	25
7	Case Study: <i>Drive Through Fire</i>	26
7.1	The film	26
7.2	The listening report	26
7.3	Descriptive output of the pipeline	27
7.4	The shape of the film	27
7.5	Dialogue-referenced measures	27
7.5.1	Loudness by the standard measures	28
7.5.2	Subtitles and the dialogue proxy	28
7.5.3	The dialogue reference	29
7.5.4	Exceedance	29
7.5.5	Occupation blocks	30
7.5.6	Two regimes	31
7.5.7	Dialogue exposed to throttling	33

7.5.8	Recovery	33
7.5.9	Swells	33
7.6	Assessment	34
8	Overdetermination	35
8.1	The idea	35
8.2	Measurement	35
8.3	Result for the case	36
8.4	What the index cannot show yet	36
8.5	A note on the earlier reading	37
8.6	Status	37
IV	Response and Intervention	38
9	The Hand on the Volume	39
9.1	Origin of the measure	39
9.2	Why volume behaviour	39
9.3	Recording	39
9.4	Annotation of candidates	40
9.5	Design	40
9.5.1	Pre-registration	40
9.5.2	Comparison corpus	41
9.5.3	Playback conditions	41
9.6	Analysis	41
9.7	Aggregation	41
10	Remixing for the Listener	42
10.1	A continuum of interventions	42
10.2	A dialogue-forward downmix	43
10.3	Limitations	43
10.4	Beyond channels	43
10.5	The objection from intent	44
10.6	Evaluation	44
V	Conclusion	45
11	What Has Been Shown	46
11.1	The argument in brief	46
11.2	Graded claims	46
11.3	Limitations	48
11.4	A programme	48
11.5	Closing	48
A	Reproducing the Results	49

B Annotation Instrument	50
B.1 Label vocabulary	50
B.2 Fields	50
C Code	51
C.1 Feature extraction and detection	51
C.2 Dialogue-referenced occupation analysis	65
C.3 Supplementary statistics	76
C.4 Blinded annotation set	79
C.5 Volume behaviour logger for <code>mpv</code>	81
References	83

Part I

The Problem

Chapter 1

Occupation, Not Loudness

1.1 A familiar complaint, stated precisely

Few complaints about home viewing are as common as the one about dialogue and explosions. The volume that makes speech intelligible makes the action unbearable; the volume that makes the action tolerable makes the speech inaudible. Viewers respond by keeping a hand on the remote. Industry responses have been largely framed in terms of loudness: broadcast loudness normalisation, legislation against loud advertisements, and dynamic range compression modes built into consumer decoders. These responses treat the problem as one of level.

This monograph takes a different view. The difficulty is not that a soundtrack is loud, nor even that it has a wide dynamic range. Both are ordinary properties of cinema sound, and both are intended. The difficulty arises from three conditions holding together:

1. the listening level is set for dialogue, because dialogue is what must be understood;
2. non-dialogue material exceeds that level by a large margin; and
3. it does so for long, nearly continuous stretches, with few intervals in which the level returns to the dialogue range.

The first condition fixes a reference. The second is a question of magnitude. The third is a question of temporal organisation, and it is the one that loudness measures summarise away. An integrated loudness figure for a film is a single number over its whole running time; a loudness range figure describes a distribution. Neither says whether the loud material arrives as occasional peaks separated by long rests or as a continuous block several minutes long. The distinction between those two cases is the subject of this book.

1.2 Peak intensity and occupation

The central distinction can be stated compactly. *Peak intensity* concerns how far above the reference a soundtrack goes. *Occupation* concerns how much of the time it stays there, and how continuously. A conventional dramatic structure has a rhythm of rest, rise, peak, and release. A soundtrack can instead proceed through rise, impact, barrage, further rise, and further impact without returning to rest. The second pattern need not have higher peaks than the first. What distinguishes it is that the listener is never released.

The practical consequence is that the listener's volume control becomes part of the soundtrack's reproduction. If the reference is set for speech and the soundtrack occupies the range well above it for minutes at a time, the listener must either accept the exceedance, reduce the volume and lose the next line of dialogue, or reduce the volume to nothing and wait. The case at the centre of this study involved the last of these. For action sequences lasting one to two minutes or more, the volume was turned fully down until the scene calmed.

1.3 Why a single case

This monograph is built around one film, *Drive Through Fire*, and one reproduction setup. That is a limitation and it is stated as such. It is also a methodological choice. The concepts needed to describe the problem (dialogue reference, exceedance, occupation block, recovery window, salience channel, throttle) are not yet standard, and the instruments needed to measure them do not exist off the shelf. A single case examined closely is the right scale at which to build and debug those instruments. The case is not offered as evidence that contemporary film mixing is generally at fault. It is offered as a worked example of how that question could be answered.

1.4 Grading claims

Every substantive claim in the book is assigned one of four grades, and Chapter 11 tabulates them.

Established. Claims supported by standards, published measurement, or well-replicated psychoacoustics, independent of this study. Example: cinema reproduction is calibrated to a reference sound pressure level; home reproduction generally is not.

Measured. Claims about the *Drive Through Fire* soundtrack that follow from the analysis pipeline as corrected, with the caveats of Chapters 4 and 5.

Hypothesised. Claims that the framework predicts and that the measurements are consistent with, but that require listener data, a comparison corpus, or measurement of the playback chain to test.

Rejected. Claims that were made during the analysis, sometimes confidently, and that did not survive inspection. These are kept because the errors are instructive.

1.5 Plan of the book

Chapter 2 surveys the standards and literatures that bear on the problem: loudness measurement, cinema calibration, film sound practice and theory, psychoacoustics, and noise annoyance. Chapter 3 gives formal definitions and states the hypotheses. Chapter 4 describes the analysis pipeline. Chapter 5 documents its failures and their corrections. Chapter 6 treats the reproduction chain as an object of study in its own right. Chapter 7 presents the case. Chapter 8 examines the co-occurrence of salience channels. Chapter 9 specifies the response measure and the study design needed to test the framework against listener behaviour. Chapter 10 develops an intervention. Chapter 11 grades the results.

Chapter 2

Background: Standards, Practice, and Perception

This chapter gathers the material the framework depends on. It is not a survey of film sound studies. It collects, from several fields that rarely cite each other, the pieces needed to state the problem of occupation precisely: how loudness is measured and regulated, how cinema reproduction is calibrated and why home reproduction differs, how film sound is organised in practice, and what psychoacoustics and noise research say about intelligibility, annoyance, and recovery.

2.1 Measuring loudness

The reference algorithm for programme loudness is ITU-R BS.1770 (ITU-R, 2015). It applies a frequency weighting (the K-weighting, a high-shelf and high-pass pair approximating the head's acoustic effect and reduced low-frequency sensitivity), computes mean-square power per channel, sums channels with weights that favour the surrounds slightly, and gates out quiet blocks so that silence does not dilute the result. Its output is expressed in LUFS (or LKFS, the same unit under a different name). European broadcast practice under EBU R 128 normalises programmes to -23 LUFS integrated loudness (EBU, 2020); North American practice under ATSC A/85 uses -24 LKFS (ATSC, 2013). In the United States the Commercial Advertisement Loudness Mitigation Act made A/85 compliance for advertisements a legal requirement (CALM Act, 2010).

Integrated loudness is a single number for a whole programme. EBU Tech 3342 supplements it with Loudness Range (LRA), a statistic of the distribution of short-term loudness values after gating, computed as the spread between the 10th and 95th percentiles (EBU, 2016). LRA answers how widely the loudness varies. It does not answer how that variation is arranged in time. A programme alternating every ten seconds between quiet and loud passages and a programme with one quiet hour followed by one loud hour can have the same integrated loudness and the same LRA. For the purposes of this study that is the decisive limitation of the standard measures: they are distributional, and occupation is a temporal property.

2.2 Dialogue as reference

Broadcast and home formats already acknowledge that dialogue is the natural anchor for listening level. The AC-3 bitstream carries a *dialnorm* parameter describing the average level of dialogue in the programme, which decoders use to bring dialogue to a common reproduction level; the same format carries dynamic range control profiles that compress loud and quiet passages toward that anchor when the listener selects a reduced-range mode (ATSC, 2018). Consumer “night mode” settings are descendants of this mechanism. Their existence is itself evidence, of an institutional kind, that the gap between dialogue level and effects level is a recognised home-listening problem rather than an idiosyncratic one.

Speech intelligibility has its own measurement tradition. The Speech Intelligibility Index (ANSI, 1997) weights frequency bands by their importance to understanding; the most heavily weighted bands lie in the region from roughly one to four kilohertz, where consonant cues are carried. A reproduction chain weak in that region reduces intelligibility at a given level, which in turn raises the level a listener must choose to follow speech. This point returns in Chapter 6.

2.3 Cinema calibration and the home

Theatrical reproduction is calibrated. Under SMPTE practice each screen channel is aligned so that a pink-noise signal at a specified digital reference level produces a specified sound pressure level at the reference listening position, conventionally 85 dB(C) for -20 dBFS (SMPTE, 2012). Mixing stages are aligned the same way. A film mix is therefore authored for a known relationship between digital level and acoustic level. Its dynamic range, including the distance between dialogue and the loudest effects, is chosen with that relationship in mind.

Home reproduction has no such alignment. The listener chooses the level, typically lower than theatrical reference, and chooses it by ear, usually for dialogue. Equal-loudness relationships are level dependent (Fastl and Zwicker, 2007), so a mix balanced at reference level is heard with a different spectral balance at lower levels. More importantly for this study, the listener sets the anchor, and the anchor is speech. Everything the mix places far above speech arrives far above the listener’s chosen level. A dynamic range that is intended in a calibrated theatre becomes, in an uncalibrated room, a sequence of exceedances that the listener manages by hand.¹

2.4 Film sound practice and theory

The film sound literature of the digital era documents the expansion of dynamic range and low-frequency capacity that multichannel formats made available (Sergi, 2004; Kerins, 2011). The low-frequency effects channel and the headroom of digital formats allow effects to be much louder, relative to dialogue, than optical soundtracks permitted.

Two ideas from practitioners and theorists are central here. The first is Chion’s observation that cinema is *vococentric*: its sound is organised around the voice, and the ordinary work of mixing is to keep speech intelligible against everything else (Chion, 1994). The complaint at

¹This is an established fact about calibration practice, not a claim that any particular film was mixed badly. A mix can be entirely correct for its intended room and still be difficult in an uncalibrated one. The monograph distinguishes these throughout.

the root of this study is, in Chion’s terms, a failure of vococentrism at the point of reproduction. The second is Murch’s account of density (Murch, 2005). Murch describes a practical limit on how many simultaneous layers of sound of the same kind an audience can follow as distinct, and a strategy of distributing layers across contrasting kinds (dialogue, music, effects of different spectral and rhythmic character) so that density does not collapse into undifferentiated mass. Murch’s concern is clarity within a moment. The present study’s concern is continuity across moments. The two are complementary: a mix can be clear in every individual moment and still occupy the listener for too long.

2.5 Psychoacoustics

Classical psychoacoustics offers measures closer to perception than digital level. Fastl and Zwicker’s models of loudness (in sones), sharpness (weighting of high-frequency energy), roughness (fast amplitude modulation), and fluctuation strength (slow modulation), and the psychoacoustic annoyance index built from them, are the standard descriptive vocabulary (Fastl and Zwicker, 2007). The analysis pipeline of Chapter 4 uses cruder spectral descriptors (centroid, flatness, zero-crossing rate), and Chapter 5 shows one consequence of that choice. Replacing them with the psychoacoustic measures is part of the programme proposed in Chapter 9.

Two further psychoacoustic facts matter. Forward masking reduces sensitivity to quiet sounds for a short interval following a loud one (Moore, 2012), so speech arriving immediately after an impact is harder to hear than the same speech after silence. And the auditory system organises sound into streams by onset, spectrum, and continuity (Bregman, 1990); abrupt onsets are strong cues for the formation of a new stream and draw attention. A soundtrack dense with abrupt onsets is, in stream terms, repeatedly asking for attention.

2.6 Noise, annoyance, and control

Community noise research establishes that annoyance grows with level, with the number of events, and with their duration (Kryter, 1985; Miedema and Oudshoorn, 2001). The finding most relevant here is older and experimental. Glass and Singer showed that the aftereffects of noise exposure on subsequent task performance depended strongly on predictability and on perceived control: participants who believed they could stop the noise showed markedly reduced aftereffects even when they did not use the control (Glass and Singer, 1972). A volume control is exactly such a means of control. Its use during a film is therefore not only a response to excessive sound but a measurable act of regaining control over it, which is part of why Chapter 9 treats volume behaviour as the natural dependent variable.

2.7 Sound, affect, and the soundscape

Two broader literatures frame the problem. Goodman’s account of sonic warfare treats low frequencies and sudden sound as instruments of affective control that operate below deliberate attention (Goodman, 2010). Schafer’s distinction between hi-fi soundscapes, in which discrete sounds are heard clearly against a quiet ground, and lo-fi soundscapes, in which signals crowd each other and perspective is lost, gives a vocabulary for what a highly occupied soundtrack does to a listening room (Schafer, 1977). Finally, the music-industry “loudness war” shows how

competitive pressure can push a production norm toward sustained high level at the expense of contrast (Vickers, 2010). Whether an analogous pressure operates in film mixing is outside this study's evidence, but the analogy explains why the question is worth asking.

2.8 The gap

None of these literatures measures the quantity this study is concerned with: the temporal structure of a soundtrack's exceedances over its own dialogue level, and the behaviour that structure forces on a listener reproducing it at an uncalibrated level through a particular chain. Loudness standards measure distributions. Film theory describes intentions and effects. Psychoacoustics and noise research describe perception and annoyance under controlled conditions. The framework in the next chapter is an attempt to connect them.

Chapter 3

A Framework for Occupation

This chapter defines the quantities used in the rest of the book and states the hypotheses they are meant to test. The definitions are deliberately simple. Each is chosen so that it can be computed from a soundtrack file with the pipeline of Chapter 4 and so that its dependence on free parameters is explicit.

3.1 Time base and level

Let the film occupy the interval $[0, T]$ and let time be discretised into one-second bins indexed by t . Let $L(t)$ denote the level of the reproduced signal in bin t , computed as the energy mean of the level values falling in that bin:

$$L(t) = 10 \log_{10} \left(\frac{1}{|F_t|} \sum_{f \in F_t} 10^{\ell_f/10} \right), \quad (3.1)$$

where F_t is the set of level measurements falling in bin t and ℓ_f is the level of measurement f . Averaging in the power domain rather than in decibels matters: a second containing one loud impact and otherwise near silence should register the impact, not the silence.

The primary level measure is BS.1770 momentary loudness (ITU-R, 2015), measured on the full 5.1 mix at ten values per second and energy-averaged into one-second bins, in LUFS. It applies the standard’s K-weighting and channel weights and, as the standard specifies, excludes the low-frequency effects channel. When the film itself is not available to the analysis, the pipeline falls back to unweighted RMS of a mono downmix of the main channels, in dBFS (Chapter 4). Only differences in level enter the framework, so the choice of unit does not affect the definitions below, but K-weighting does change which material counts as loud: it discounts low frequencies and emphasises the presence region, closer to how loudness is perceived.

A bin is *audible* if $L(t)$ exceeds a floor $L_{\min}$ (default -60 LUFS, or -65 dBFS for the RMS fallback). Inaudible bins are excluded from all statistics that are normalised by duration.

3.2 The dialogue reference

Two definitions of dialogue time are used. The primary one is taken from the film’s subtitles, which record when speech occurs. The secondary one is a centre-channel proxy, which requires nothing but the audio; it is retained because most films in a comparison corpus will have it

and not all will have reliable subtitles, and because comparing it with the subtitle definition measures how far the proxy can be trusted.

Definition 3.1 (Subtitled dialogue bins). Let a subtitle file contain cues (a_i, b_i, x_i) with start and end times and text. Remove cues whose text describes sound rather than speech: text consisting only of bracketed or parenthesised descriptions such as [GUNFIRE] or (sighs), or of music symbols. Let $\kappa(t)$ be the fraction of bin t covered by the remaining cues after shifting all cue times by an offset ω_s . The set of subtitled dialogue bins is

$$\mathcal{S} = \{t : t \text{ audible}, \kappa(t) \geq 0.5\}. \quad (3.2)$$

The offset ω_s corrects for subtitles timed to a different release of the film. It may be given, or estimated as the lag, within ± 30 s, that maximises the correlation between cue coverage and centre-channel dominance.

Definition 3.2 (Dialogue-likely bins). Let $C(t)$ and $N(t)$ be the levels of the centre channel and of the non-centre mix (front and surround pairs) in bin t . The set of dialogue-likely bins is

$$\mathcal{D} = \{t : t \text{ audible}, C(t) - N(t) \geq c, \tau(t) \leq Q_p(\tau \mid C - N \geq c)\}, \quad (3.3)$$

where c is a centre-advantage threshold (default 6 dB), $\tau(t)$ is transient activity (Section 3.5), and Q_p is its p -th percentile (default 75) among centre-dominant audible bins.

The definition encodes a convention of film mixing, that dialogue is carried predominantly in the centre channel, and removes the most transient-heavy centre-dominant bins because centre-panned impacts and gunfire are common in action scenes. It remains a proxy. Music, effects, and ambience can sit in the centre, and dialogue can be spread. Its agreement with $\mathcal{S}$ is reported as precision $|\mathcal{D} \cap \mathcal{S}|/|\mathcal{D}|$ and recall $|\mathcal{D} \cap \mathcal{S}|/|\mathcal{S}|$.

Definition 3.3 (Dialogue reference). The dialogue reference is $D = \text{median}\{L(t) : t \in \mathcal{S}\}$ when subtitles are available, and $D = \text{median}\{L(t) : t \in \mathcal{D}\}$ otherwise.

Subtitles mark when speech occurs, not how loud it is, and they mark shouted lines inside gunfights as readily as quiet conversation. The median is robust to such lines because most dialogue in most films is not delivered over gunfire. As a check, the reference is also computed with the most transient-heavy quarter of subtitled bins removed.

D is the level at which a listener who sets the volume for speech is anchored. It is internal to the film, and that is intentional: the listener's anchor is also internal to the film.

3.3 Exceedance and occupation

Definition 3.4 (Exceedance). $E(t) = L(t) - D$.

Definition 3.5 (Occupation). For a tolerance $\delta > 0$ (default 10 dB), bin t is *occupied* if $E(t) \geq \delta$. The occupation fraction is

$$\phi_\delta = \frac{|\{t \text{ audible} : E(t) \geq \delta\}|}{|\{t \text{ audible}\}|}. \quad (3.4)$$

Definition 3.6 (Occupation block). Let O be the set of occupied bins. Form maximal runs of consecutive bins in O , then merge any two runs separated by a gap of at most g seconds (default 5). An *occupation block* is a merged run of total length at least m seconds (default 60).

The merging step reflects the fact that a two-second dip below $D + \delta$ in the middle of a firefight does not give the listener a usable opportunity to restore the volume. The minimum length m reflects the case: the behaviour at issue is reducing the volume for a minute or more.

A block need not be occupied throughout. Its internal composition is summarised by the share of occupied bins and by percentiles of E within it.

3.4 Recovery

The pipeline of Chapter 4 defines recovery relatively, as intervals of at least five seconds in which a composite burden score lies in its own bottom quartile. Chapter 5 explains why that definition cannot support claims about how much recovery a film provides: roughly a quarter of the film is below its own lower quartile by construction. A relative definition can still locate the longest stretches without recovery, and it is used for that purpose only.

An absolute definition, anchored to the dialogue reference, is preferred:

Definition 3.7 (Recovery window). For a margin ρ and width w , a recovery window is an interval of at least w consecutive audible bins in which $E(t) \leq \rho$ throughout. The default is $\rho = 0$, $w = 5$: at least five seconds at or below dialogue level.

Definition 3.8 (Recovery gap and recovery debt). A recovery gap is the interval between the end of one recovery window and the start of the next. Within a gap beginning at t_0 , the recovery debt at time t is

$$K(t) = \sum_{s=t_0}^t \max(0, E(s) - \delta). \quad (3.5)$$

A recovery gap is an interval without a qualifying recovery window. It is not an interval of quiet: a gap may contain many seconds at or below the dialogue reference (in runs shorter than w), and most of its seconds may be occupied.

K is a crude accumulator: it grows only while the soundtrack exceeds tolerance and resets only when the soundtrack returns to dialogue level for long enough. It is proposed as a predictor, not a model of physiology.

3.5 Salience channels

Level is one way a soundtrack claims attention. The pipeline measures four others, each a candidate channel of salience:

Transient activity $\tau(t)$. Density of broadband spectral-flux onsets over a five-second window, weighted by audibility.

Low-frequency onset activity $\lambda(t)$. Density of onsets in the band below 250 Hz over a four-second window. This is often percussion but also engines, impacts, and explosions; it is named for what it measures.

Swell events σ . Sustained rises in level of at least 6 dB over windows of two to eight seconds that are not carried by a single step.

Spectral harshness $\eta(t)$. A composite of spectral centroid, flatness, and zero-crossing rate, weighted by audibility.

Definition 3.9 (Overdetermination index). Given thresholds θ_k for each channel, $\Omega(t) = \sum_k \mathbf{1}[\text{channel } k \text{ elevated at } t]$, with level exceedance $E(t) \geq \delta$ counted as one of the channels.

Ω counts how many independent means the soundtrack is using at once to claim attention. The name reflects the hypothesis that high Ω is redundant signalling, several mechanisms communicating the same instruction to attend.

3.6 A listener model

Let the listener set the volume so that dialogue is reproduced comfortably, and let $G(t) \leq 0$ be the additional gain applied by the listener at time t relative to that setting. Suppose the listener tolerates reproduced level up to $D + \delta$. Holding a block at tolerance requires $G(t) \leq -\max(0, E(t) - \delta)$. Since listeners do not adjust gain second by second, a single setting per block is more realistic. The *implied throttle depth* of a block b is

$$T_b = \max(0, Q_{95}(E | b) - \delta), \quad (3.6)$$

the attenuation needed to hold the block's 95th-percentile exceedance at tolerance.

Two responses are available when T_b is large. The listener can attenuate by T_b and lose the dialogue inside the block, which sits at $D + G < D - T_b$, or can turn the volume fully down and wait. The model predicts the second response when T_b is large and the block is long, because attenuation by a large T_b leaves nothing intelligible to attend to anyway. This is the behaviour reported in the case.

3.7 Hypotheses

Hypothesis 1 (Occupation over level). Volume reductions are better predicted by occupation variables (current $E(t)$ together with time already spent in the current block) than by instantaneous level alone, and far better than by whole-programme statistics such as integrated loudness or loudness range.

Hypothesis 2 (Recovery debt). Conditional on $E(t)$, the probability of a volume reduction increases with $K(t)$.

Hypothesis 3 (Overdetermination). Conditional on $E(t)$, the probability of a volume reduction increases with $\Omega(t)$.

Hypothesis 4 (Chain amplification). A reproduction chain that is weak in the speech-importance band and that saturates on transients increases effective exceedance: it raises the level needed for intelligible dialogue and adds distortion products during the loudest passages.

Hypothesis 5 (Intervention). A dialogue-forward downmix with dynamic range compression reduces the number and duration of volume reductions without reducing comprehension of the dialogue.

None of these is tested in this monograph. Chapter 7 measures the soundtrack-side quantities they depend on. Chapter 9 specifies the data needed to test them.

Part II

Instruments

Chapter 4

The Analysis Pipeline

The quantities of Chapter 3 are computed in two stages. The first, `analyze-soundtrack-v3.1.py`, decodes the film, extracts frame-level features in bounded memory, derives detector scores and events, and writes audition clips. The second, `occupation.py`, measures BS.1770 loudness in its own decode pass, aligns the film’s subtitles, aggregates everything to one-second bins, and computes the dialogue-referenced quantities. Both are listed in Appendix C. This chapter describes what they compute and why, and records the parameters that the results depend on.

4.1 Design constraints

A feature film at 48 kHz with six channels is roughly 1.5 billion samples. The first attempt at this analysis computed a short-time Fourier transform of the whole film in memory and was terminated by the operating system. Every later version follows three rules. Decoding and channel manipulation are done by `ffmpeg`, which streams. Python reads the decoded analysis stems in chunks of sixty seconds and keeps only compact per-frame features. Normalisation statistics are computed only after all chunks have been processed, so that a quiet minute and a loud minute are compared against the same film-wide scale rather than each being treated as its own acoustic universe.

4.2 Analysis stems

Four single-channel stems are produced:

Broadband. A mono downmix of the full mix at 16 kHz.

Low band. The same downmix low-pass filtered at 250 Hz, at 8 kHz.

Centre. The front centre channel alone, at 16 kHz.

Non-centre. An equal-weight mix of the front left and right and the two surround channels, at 16 kHz.

One property of the downmix matters for interpretation. When a stream is tagged with a 5.1 layout, the default `ffmpeg` downmix to mono does not include the low-frequency effects channel. This was verified directly: a test file with a 50 Hz tone in the LFE channel alone

produced output at -91 dB, while the same tone in the centre channel produced output at -19.7 dB. Consequently the “low-frequency” measures in this study describe bass carried in the main channels, not the LFE channel. Many software players make the same omission when downmixing for stereo output, so the analysed signal may resemble what a listener on a stereo or mono chain actually hears more closely than a full-range measurement would; this is an assumption to be checked against the player used, not a finding.

4.3 Frame features

Features are computed with `librosa` (McFee et al., 2015) at a hop of 512 samples (32 ms at 16 kHz):

Level. Frame RMS in dBFS.

Onset strength. Spectral flux on a mel spectrogram in the log domain, the standard onset detection function (Bello et al., 2005).

Spectral shape. Centroid, flatness (Johnston, 1988), and zero-crossing rate.

Low-band level and onset strength on the low stem.

Centre and non-centre level.

Onset strength, centroid, flatness, and zero-crossing rate are all nearly independent of absolute level. A click at -70 dBFS produces the same log-domain spectral flux as the same click at -30 dBFS; the noise floor of a near-silent passage has maximal flatness. This property is the source of the most consequential failure described in Chapter 5.

4.4 Audibility weighting

Version 3.1 multiplies every shape and transient feature by an audibility weight

$$w(t) = \text{clip}\left(\frac{L(t) - L_{\min}}{L_{\text{full}} - L_{\min}}, 0, 1\right), \quad (4.1)$$

with $L_{\min} = -65$ and $L_{\text{full}} = -45$ dBFS by default, and a corresponding weight on the low band with bounds -70 and -50 dBFS. Robust statistics (median and scaled median absolute deviation, with interquartile and standard-deviation fallbacks when the MAD is degenerate) are computed over audible frames only.

4.5 Detectors

Impact. Peaks of a weighted sum of standardised onset, level, and spectral shape, above the 98.5th percentile and at least 0.75 s apart.

Low-frequency onset activity. Four-second moving density of positive standardised low-band onset strength, combined with low-band level; peaks above the 97.5th percentile at least 4 s apart.

Harshness. Weighted sum of standardised centroid, flatness, zero-crossing rate, and level; peaks above the 98th percentile at least 3 s apart.

Barrage. Five-second moving density of positive standardised onset strength, combined with level and centroid; contiguous regions above the 85th percentile lasting at least 3 s.

Burden. A bounded composite of normalised level, onset strength, low-frequency onset density, centroid, flatness, and barrage density, smoothed over half a second. High-load regions are those above its 80th percentile for at least 4 s; relative recovery regions are those below its 25th percentile for at least 5 s.

Swell. An event detector described in Chapter 5.

Every detector threshold except the swell detector is a percentile of the film’s own score distribution. The consequences of that choice for interpretation are taken up in Chapter 5. The dialogue-referenced quantities of Chapter 3 were introduced to avoid them.

4.6 Audition clips

For each of the swell, low-frequency, impact, harshness, and barrage detectors, the twenty highest-scoring candidates are cut from the source as short stereo MP3 clips, with a playlist per category and an annotation table with blank label and notes columns. Point events are cut from twelve seconds before to eight seconds after; swells from four seconds before the rise to six seconds after it; regions from five seconds before their start. This is the bridge from acoustic proxies to identified sources, and Chapter 9 describes how it must be modified before its labels can count as evidence.

4.7 Loudness

The level measure of Chapter 3 is taken from `ffmpeg`’s `ebur128` filter, an implementation of BS.1770 and EBU R 128, run once over the film’s full 5.1 stream. The filter reports momentary loudness (400 ms window) ten times a second; these values are energy-averaged into one-second bins. The same pass yields the film’s integrated loudness and its loudness range under EBU Tech 3342, which Chapter 7 sets beside the occupation measures. Both are cached so that later runs do not decode the film again.

The RMS level of the version 3.1 frame table remains available and is used when the film is not supplied. The two measures differ in weighting, channel treatment, and window, and they are not interchangeable in absolute terms; within one analysis only one is used.

4.8 Subtitles

The film’s subtitle file (SubRip format) is parsed into timed cues. Formatting tags are stripped and HTML entities decoded. Cues whose text is entirely descriptive of sound, such as `[GUNFIRE]`, `(sighs)`, or music symbols, are removed, since subtitles for deaf and hard-of-hearing viewers use them to describe non-speech sound; cues that mix a description with speech, or carry a speaker label before speech, are kept. Each second receives the fraction of its duration covered by remaining cues.

Subtitle files are often timed to a different release of a film, shifted by a few seconds. The script estimates a constant offset by computing the correlation between cue coverage and a binary indicator of centre-channel dominance at lags from -30 to $+30$ s in half-second steps, and applies the best lag only if it improves the correlation by more than ten percent over no shift. A fixed offset can be supplied instead. Timing that drifts over the film, as when a subtitle file was made for a version at a different frame rate, is not corrected by a constant offset; the reported correlation at the chosen lag is a check on this.

4.9 Parameters

Table 4.1 records the defaults. Every result reported in Chapter 7 uses them unless stated otherwise.

Parameter	Role	Default
Chunk	Decode and feature window	60 s
Sample rate	Broadband, centre, non-centre stems	16 kHz
Hop	Frame spacing	512 samples
$L_{\min}$, L_{full}	Audibility weight bounds (frame features)	-65 , -45 dBFS
Audibility floor	Per-second bins, BS.1770 level	-60 LUFS
Low-band bounds	Low-band audibility weight	-70 , -50 dBFS
Swell windows	Rise windows	2, 4, 6, 8 s
Swell minimum rise		6 dB
Subtitle coverage	Minimum cue coverage for a subtitled bin	0.5
Subtitle offset search	Lag range and step	± 30 s, 0.5 s
c	Centre advantage for dialogue-likely bins	6 dB
p	Transient cap percentile for dialogue-likely bins	75
δ	Occupation tolerance	10 dB
g	Block merge gap	5 s
m	Minimum reported block	60 s
ρ , w	Absolute recovery margin and width	0 dB, 5 s
Channel threshold	Elevated salience channel	1.5 (standardised)
Residual margin	Share of threshold left after level fit	0.5

Table 4.1: Pipeline parameters.

Chapter 5

How the Detectors Failed

This chapter documents four failures of the analysis as it was first built and first read. Each produced output that looked meaningful. Each was caught only by comparing one plot against another or by running the pipeline on a synthetic signal whose contents were known. They are recorded in detail because they are generic: any analysis that standardises heavy-tailed features, uses level-invariant descriptors, or thresholds by percentiles is exposed to them.

5.1 The swell detector that detected transients

Version 3 attempted to detect gradual swells by smoothing the level trace, taking its positive gradient, standardising the result by median and median absolute deviation, smoothing again, and subtracting a penalty proportional to positive standardised onset strength so that abrupt impacts would not count as swells.

On the film, more than half of the positive-gradient values were exactly zero, because at any moment the smoothed level is as likely to be falling as rising and every falling frame is clipped to zero. The median was therefore zero and so was the median absolute deviation. The standardisation function returned an array of zeros when the scale was degenerate. The gradient term vanished, and what remained was

$$\text{swell}(t) = -0.25 \max(0, z_{\text{onset}}(t)), \quad (5.1)$$

an inverted, scaled copy of the onset trace. Figure 5.1 shows the result. The deepest troughs of the “swell” plot, near -14 at about 14 and 63 minutes, sit exactly under the two largest onset peaks. Every swell candidate reported by version 3 was a peak of this inverted function, which is to say a place where nothing happened.

The failure is data dependent. On a synthetic test file the same code did not degenerate, because the synthetic signal had fewer exact ties. A test that passes on synthetic data does not certify the code on real data, and the converse also holds; both are needed.

Version 3.1 replaces the frame-level gradient with an event detector. Level is averaged in the power domain over 0.25-second bins and median-filtered over about one second, so that a single transient cannot carry a bin. For each candidate window $\Delta \in \{2, 4, 6, 8\}$ seconds the rise $S(t, \Delta) = L(t + \Delta) - L(t)$ is computed. A candidate is accepted if the rise is at least 6 dB, at least 60% of the bin-to-bin steps within the window are upward, the two largest steps together account for no more than half the rise, and the window ends above the audibility floor. The

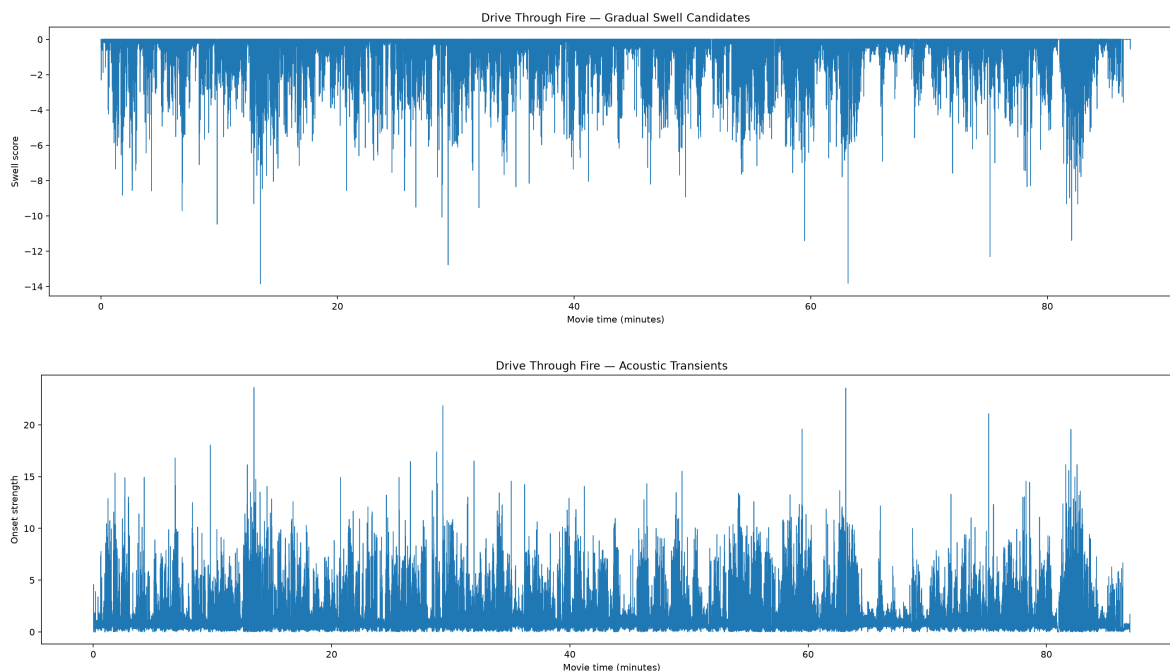


Figure 5.1: Version 3 swell score (top) and onset strength (bottom). With a zero MAD the swell score reduced to a negated multiple of positive standardised onset strength; its troughs mirror the onset peaks.

two-largest-steps rule was added after a synthetic test showed that a hard cut from near silence into a loud passage, smeared by the analysis window across two bins, otherwise passed as a swell. On the synthetic file the corrected detector recovered a ramp placed between 150 and 158 seconds as an event from 149.25 to 157.25 seconds and reported nothing else.

The standardisation function was also changed so that a degenerate MAD falls back to the interquartile range and then to the standard deviation, rather than silently returning zeros.

5.2 Harshness in silence

The version 3 harshness score combined standardised spectral centroid, flatness, zero-crossing rate, and level. Its largest values in the entire film, near 66 against a typical ceiling around 30, occur at the very start, at about 81 minutes, and at the very end (Figure 5.2). Each coincides with a dropout to digital silence at -100 dBFS on the level plot. In near silence the spectrum is the noise floor, which is spectrally flat and dominated by high-frequency dither; flatness and zero-crossing rate are maximal and the centroid is high. The most “harsh” moments of the film were the moments with no sound in them.

The same level invariance affected every onset-based measure. Between about 81 and 84 minutes the level trace sits mostly between -55 and -75 dBFS, far below the rest of the film, yet transient, barrage, low-frequency, and burden scores all peak there. An early reading of the plots described that stretch as the most intense passage of the film. It was a quiet passage with dense low-level transients. The subtitle timings introduced later (Chapter 7) identify it as the film’s closing dialogue scene: more than half of its seconds carry subtitled speech, and

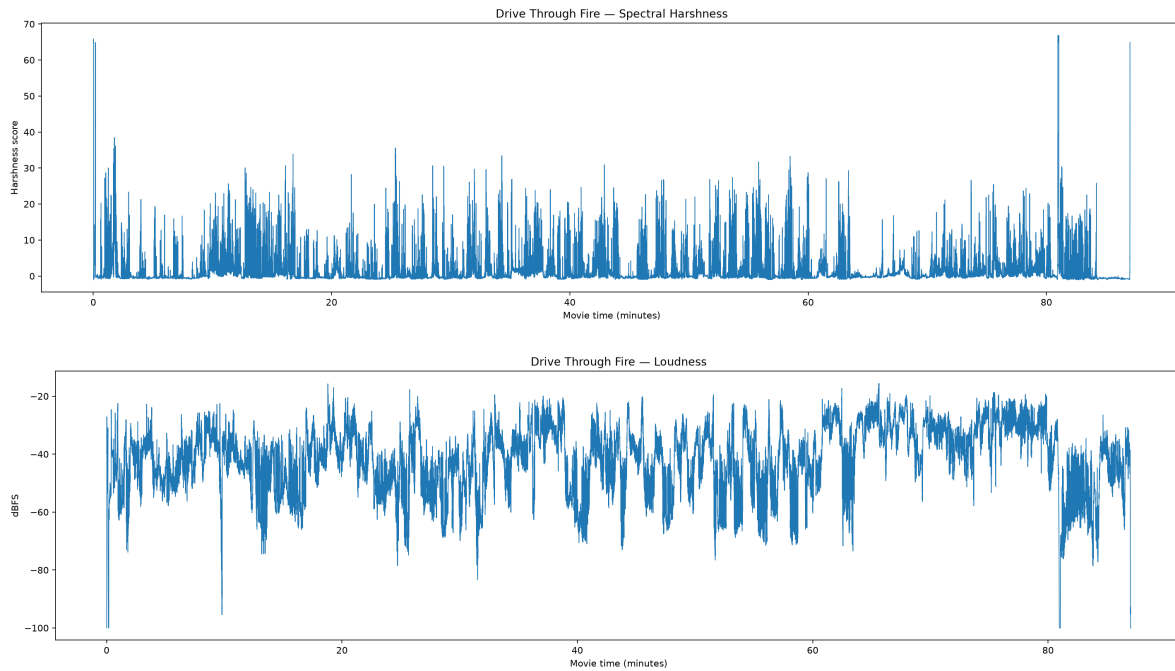


Figure 5.2: Version 3 harshness (top) and level (bottom). The three largest harshness spikes coincide with dropouts to -100 dBFS.

its median level lies about 10 dB below the film’s dialogue reference. The end credits begin only after about 84 minutes.

A synthetic test made the magnitude of the problem concrete. A passage of short noise bursts at a mono level of about -83 dBFS received a mean version 3 barrage score of 87, against 26 for a passage of modulated speech-band noise at about -50 dBFS, and a higher burden (0.54 against 0.50). After audibility weighting (Chapter 4) the quiet passage scored -7 on barrage and 0.00 on burden, and the harshness score in digital silence fell to zero.

5.3 Statistics fixed by their thresholds

Version 3.1 on the film reported, among other things, that barrage regions occupied 8.0% of the running time, high-load regions 15.8%, and recovery regions 19.2%, and that recovery was therefore scarce. These numbers are largely determined before any audio is analysed. High load is defined as the top 20% of the burden score, so about 20% of frames exceed the threshold by construction; the 15.8% is the part of that 20% that falls in runs of four seconds or more. Recovery is the bottom 25%, so about a quarter of the film is recovery by definition; 19.2% is the part in runs of five seconds or more. Barrage is the top 15% of its score, of which 8.0% survives the three-second minimum. Similarly, the 902 impact candidates are peaks above the 98.5th percentile at least 0.75 s apart; a different percentile gives a different count.

What these statistics can legitimately show is the arrangement in time of each category: where the qualifying windows fall, and the distribution of run lengths and gaps. The longest interval with no relative recovery window, 694.7 seconds, is such a statistic, and it is informative about the location of the film’s densest stretch. It cannot show how much recovery the film

provides compared with other films, because a film that was relentless from beginning to end would still be one quarter “recovery”. The dialogue-referenced definitions of Chapter 3 were introduced for this reason.

5.4 Interpretive overreach

The fourth failure was not in the code. An interpretive pass over the version 3 and 3.1 plots, produced with the assistance of a language model, made a series of claims that the analysis could not support: that the 81 to 84 minute stretch was the most intense in the film (it was quiet; see above); that the film’s recovery was scarce (a threshold artefact); that the film “has plenty of dynamic range” (the analysis measured unweighted RMS of a mono downmix, not loudness range in the standard sense); that swells of 30 to 36 dB demonstrated dramatic musical rises (the detector at that point checked only the end level of a swell, so a large rise could begin in near silence and could be a scene transition); and that the low-frequency detector found drums (it measures low-band onset density from any source).

Each claim was plausible, fluent, and consistent with the listener’s report, which is what made it dangerous. The general lesson is that agreement between an analysis and a prior impression is weak evidence that the analysis is right. The specific corrections are built into the pipeline and into the reporting rules of Chapter 7: swell events carry their start and end levels; low-frequency activity is named for what it measures; percentile-defined totals are not reported as findings; and the loudness range of the film is reported only as measured under BS.1770 and EBU Tech 3342 (Chapter 7).

5.5 Rules adopted

1. Name each detector for the quantity it measures, not for the source it is hoped to find.
2. Test each detector on a synthetic signal with known contents, and inspect its output on real data against at least one independent trace.
3. Weight level-invariant features by audibility before standardising them, and compute statistics over audible frames.
4. Treat any total produced by a percentile threshold as a description of the threshold unless it is compared across films with a common threshold.
5. Report the start and end levels of every rise.
6. Do not accept an interpretation because it agrees with the listening report.

Chapter 6

The Reproduction Chain

A soundtrack file is not what a listener hears. Between the file and the ear lie a decoder, a downmix, a volume control, an amplifier, a loudspeaker, and a room. Film sound analysis usually treats this chain as transparent or as the listener's problem. This chapter treats it as part of the object of study, because in the case at hand it is likely to have made a difficult mix considerably worse, and because the same is probably true, in milder forms, of much ordinary home viewing.

6.1 The chain in the case

The film was played on a desktop computer. The computer's front-panel headphone output was connected by a 3.5 mm cable to a small solid-state bass practice amplifier, an Orange Crush 25BX, which has a single 8-inch driver and no separate high-frequency driver. The listener controlled level with the computer's volume control, which sits before the amplifier's input stage.

Several properties of the chain are not known and are recorded as open variables:

- the media player, and therefore the downmix it applied to the 5.1 stream, including whether the LFE channel was included;
- which amplifier input received the signal (instrument input or an auxiliary input, if present), and therefore whether both channels of the headphone output were summed or only one was used;
- the amplifier's gain, equaliser, and master settings;
- the listening distance and room.

6.2 Four transformations

6.2.1 Channel reduction

Six channels become one loudspeaker. A player downmixing to stereo typically places the centre channel in both left and right at reduced level, together with the corresponding surrounds. If the stereo output then feeds a mono instrument input through a standard two-conductor plug,

only the left channel reaches the amplifier and the right output is shorted to ground. In that case content panned to the right is lost, and dialogue is heard only through its share of the left channel. If the input sums both channels, material that is out of phase between left and right partially cancels. Either way the spatial separation that a multichannel mix uses to keep dialogue distinct from effects is removed. Separation by direction, one of the means by which a mix keeps several layers intelligible at once, is unavailable.

6.2.2 Band limitation

A single 8-inch driver in a small cabinet reproduces little of the lowest octave and has limited high-frequency extension. The important consequence is not the missing sub-bass but the relative treatment of the speech-importance band. The Speech Intelligibility Index weights most heavily the region from roughly one to four kilohertz (ANSI, 1997). A bass instrument amplifier is voiced for the range where bass guitar fundamentals and low harmonics lie and is not designed to reproduce consonant cues clearly. Effects and music with substantial energy between 100 Hz and 1 kHz (impacts, engines, drums, low brass) are reproduced strongly. The effect is to lower dialogue intelligibility at a given level while leaving much of the energy of action material intact.

6.2.3 Level-dependent distortion

An instrument input is designed for the output of a passive bass pickup. A computer headphone output at a high volume setting can exceed that input's clean range, and the loudest passages of a film mix are exactly the ones that will do so. The amplifier's preamplifier then clips on transients, adding harmonic distortion products that are not in the file. The loudspeaker can also reach its excursion limit on strong low-frequency content. Both effects scale with level, so they concentrate where exceedance is already largest. Part of what a listener hears as harshness in such passages may be produced by the chain.¹

6.2.4 Coupling of volume and distortion

Because the listener's volume control precedes the amplifier, turning the volume down reduces both reproduced level and preamplifier drive. A reduction therefore relieves two problems at once. This is relevant to Chapter 9: in this chain, a logged volume reduction is a response to the combined effect of level and distortion, and the two cannot be separated behaviourally without the measurements below.

6.3 How the chain enlarges exceedance

Combining these transformations with the listener model of Chapter 3 gives a specific prediction. Let D_{file} be the dialogue reference measured on the file and D_{heard} the level at which the listener must set the chain for dialogue to be intelligible through it. Band limitation and channel reduction make intelligibility harder, so D_{heard} is set higher relative to the file than it

¹This is a mechanism, graded as hypothesised. It predicts that measured distortion in the chain's output rises sharply above a particular input level, and that recordings of the chain playing the film's loudest passages contain harmonic content absent from the source. Both predictions can be tested without rewatching the film.

would be on a neutral system. Action material, reproduced strongly and with added distortion, then exceeds that higher setting by at least as much as before, and the effective exceedance

$$E_{\text{heard}}(t) = L_{\text{heard}}(t) - D_{\text{heard}} \quad (6.1)$$

is at least as large as the file-level $E(t)$ and plausibly larger. The chain does not create the gap between dialogue and action, which is a property of the mix. It widens the gap and adds distortion to the loud side of it. This is the precise content of the observation that the amplifier “exaggerated the bad mix”.

6.4 Measuring the chain

The chain can be characterised without watching the film again.

1. **Record the configuration.** Player and its audio output settings, the amplifier input used, all amplifier control positions, the computer volume setting, the cable, and the listening position.
2. **Frequency and impulse response.** Play an exponential sine sweep through the chain and record it at the listening position. Deconvolution yields the linear impulse response and separates harmonic distortion products in time (Farina, 2000). A measurement microphone is preferable; a phone gives an approximate curve.
3. **Clipping onset.** Repeat the sweep at stepped computer volume settings and find the setting at which distortion products rise sharply.
4. **Absolute calibration.** Play a reference tone at a known digital level and read the sound pressure level at the seat with a meter. This converts dBFS in the file to dB SPL at the listener, and allows recovery windows and tolerances to be expressed in absolute rather than film-relative terms.
5. **Excerpt recordings.** Record the chain playing the audition clips (Chapter 4) at the volume setting normally used, and compare each recording with its source.

6.5 Simulating the chain

With an impulse response and a clipping model, the heard signal can be approximated from the file: apply the player’s downmix, apply the listener’s gain, apply a static nonlinearity with the measured onset, and convolve with the measured impulse response (`ffmpeg`’s `afir` filter performs the convolution). Running the pipeline of Chapter 4 on the simulated signal and on the file gives two sets of occupation measures for the same film. Their difference is a measurement of what the chain adds. Until that is done, statements about the chain in this book are graded as hypothesised.

Part III

The Case

Chapter 7

Case Study: *Drive Through Fire*

7.1 The film

Drive Through Fire is an action thriller directed by Lauren Bond and Jim Carlson, with Carlson and Michael Jai White in leading roles, released to video on demand and digital platforms on 11 September 2026 (ComingSoon, 2026). Its published synopsis promises escalating violence ending in a rescue mission and a final confrontation, which is to say a genre in which extended gunfire and fight sequences are expected. Nothing in this chapter is a judgement of the film as a work. The question is how its soundtrack is organised in time relative to its dialogue, and what that organisation does to a listener at home.

The analysed file carries a six-channel 5.1 stream at 48 kHz. The decoded analysis stems run for 1:27:01.98, yielding 163,188 analysis frames.

7.2 The listening report

The report on which the case rests was given after a single viewing through the chain described in Chapter 6. It makes four claims.

- R1.** Dialogue was mixed lower than the sound effects, the swelling music, and the drums, and the difference was large enough to be comic.
- R2.** There were very long sequences of automatic gunfire and fighting.
- R3.** During those sequences the volume was reduced repeatedly, and in some cases turned fully off for one to two minutes or more, until the action on screen subsided.
- R4.** The amplifier exaggerated the imbalance.

The report is retrospective and has no timestamps. It is the observation that the rest of the book tries to explain and to make testable, not evidence for the framework. R1 and R2 are claims about the soundtrack and can be checked against the file. R3 is a claim about behaviour; the file can show whether the conditions the framework associates with such behaviour are present, but not that they caused it. R4 is a claim about the chain and is addressed only by the protocol of Chapter 6.

7.3 Descriptive output of the pipeline

Table 7.1 gives the summary produced by version 3.1. Following the rules of Chapter 5, its totals are reported as descriptions of the detectors at their thresholds, not as findings about the film.

Quantity	Value	Status
Frames below audibility floor (-65 dBFS)	3.2%	measured
Impact candidates	902	threshold-determined
Low-frequency onset candidates	52	threshold-determined
Swell events	78	measured (event criteria)
Harshness candidates	366	threshold-determined
Barrage regions (time share)	54 (8.0%)	threshold-determined
High-load regions (time share)	77 (15.8%)	threshold-determined
Relative recovery regions (time share)	78 (19.2%)	threshold-determined
Median interval without relative recovery	17.2 s	arrangement
Longest interval without relative recovery	694.7 s	arrangement

Table 7.1: Version 3.1 summary for the film. “Arrangement” marks statistics that describe where threshold-defined events fall in time, which is informative within the film even though the totals are not.

Two arrangement statistics carry information. The median interval between relative recovery windows is 17.2 seconds: over most of the film, the soundtrack returns to its own quieter states frequently. The longest interval is 694.7 seconds, more than eleven and a half minutes. The distribution is therefore strongly skewed. Recovery is ordinarily frequent, and then for one long stretch it is absent.

7.4 The shape of the film

The version 3 traces, read with the caveats of Chapter 5, show the arrangement that the skewed gap distribution implies. For the first hour the level trace alternates between passages near -40 to -60 dBFS and shorter excursions toward -25 , with frequent relative recovery windows. From about 61 minutes the level rises and stays high, mostly between -20 and -40 dBFS, until about 80 minutes. On the version 3 recovery plot (Figure 7.1) the relative recovery windows thin out after about 33 minutes and are almost absent between about 69 and 80 minutes. Low-frequency onset activity recurs throughout the film rather than being confined to its climax, with concentrations near 13 to 16, 38 to 45, 58 to 64, and 71 to 83 minutes.

7.5 Dialogue-referenced measures

The measures of Chapter 3 were computed by `occupation.py` from the version 3.1 frame table, the film’s 5.1 stream, and its subtitle file. Level is BS.1770 momentary loudness, energy-averaged per second, in LUFS.

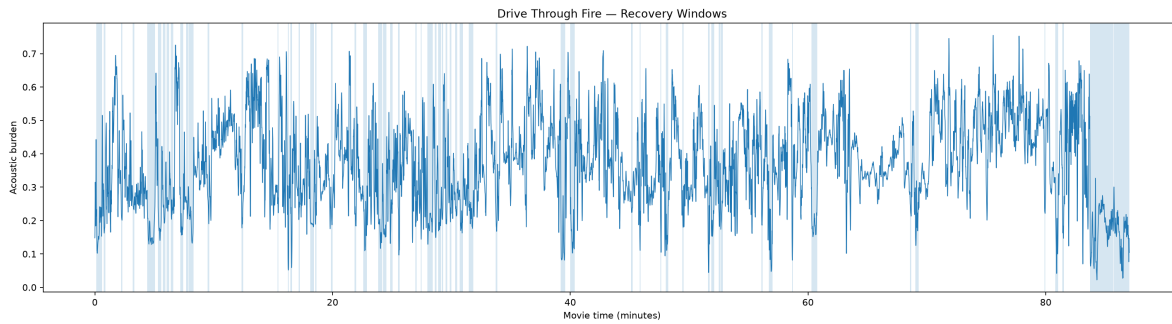


Figure 7.1: Version 3 burden with relative recovery windows shaded. The windows are frequent early, sparse after about 33 minutes, and nearly absent between about 69 and 80 minutes. Burden here is not audibility-weighted; the shaded region after about 84 minutes is the end credits.

7.5.1 Loudness by the standard measures

By BS.1770 the film’s integrated loudness is -21.6 LUFS and its loudness range under EBU Tech 3342 is 22.3 LU, spanning -39.2 to -17.0 LUFS. These are the numbers a loudness-compliance check would report. They are given first because the rest of this section describes what they leave out: where in the film the loud material sits relative to the dialogue, and how long it stays there. Two relations are worth noting in advance. The dialogue reference found below lies 13.0 dB below the integrated loudness, so a playback system that normalised this film to a loudness target would place its dialogue that far below the target. And the upper bound of the loudness range, -17.0 LUFS, sits almost exactly where the plateau of loud material described below tops out.

7.5.2 Subtitles and the dialogue proxy

The subtitle file contains 799 cues, of which 0 describe sound rather than speech and were removed, leaving 799. The applied offset is +0.0 s; the correlation between cue coverage and centre-channel dominance is 0.60 with no shift and 0.60 at the applied offset. After alignment, 1499 audible seconds are subtitled dialogue. No cue consisted solely of a sound description, so the file is an ordinary subtitle track rather than one written for deaf and hard-of-hearing viewers, and the best lag was zero: the subtitles were already aligned with this release.

Against these seconds, the centre-channel proxy (1509 seconds) has a precision of 52.0% and a recall of 52.4%. Precision measures how often the proxy’s dialogue seconds are in fact subtitled speech; recall measures how much subtitled speech the proxy finds. The dialogue reference computed from the proxy is -35.0 LUFS, against -34.6 LUFS from subtitles, and -35.6 LUFS from subtitles with the most transient-heavy quarter removed. The three agree within about one decibel, so the results that follow do not depend on which definition of dialogue is used.

The combination is instructive. The proxy identifies dialogue seconds poorly, with precision and recall both near one half, yet it estimates dialogue *level* almost exactly. Its errors are of two kinds that cancel in the median: centre-dominant seconds between cues (pauses, reactions, centre-panned ambience) that are at dialogue level without being subtitled, and subtitled

seconds excluded by the transient cap or by a momentary loss of centre dominance. For a comparison corpus this is useful: films without subtitles can be given a dialogue reference from the proxy with some confidence, but not a dialogue timeline. The film’s dialogue is strongly centred: the median centre advantage is 16.0 dB in subtitled seconds against -2.1 dB elsewhere, which is the property the intervention of Chapter 10 relies on.

7.5.3 The dialogue reference

The reference used below is taken from subtitle-defined dialogue seconds: $D = -34.6$ LUFS, computed from 1499 seconds (28.7% of the running time). The median centre-channel level in those seconds is -36.9 dBFS. The reference is a median, and dialogue itself spans a wide range around it: the 10th and 90th percentiles of subtitled seconds lie at -9.3 and +10.1 dB relative to D , the quartiles at -4.8 and +4.9 dB. Some of that spread is the dialogue and some is whatever else plays in the same second; the per-second level is the level of the whole mix. A listener anchored to dialogue is anchored to a band, not a line.

7.5.4 Exceedance

Table 7.2 gives the share of audible seconds whose level exceeds the dialogue reference by 5, 10, 15, and 20 dB, and Figure 7.2 their distribution. The 95th percentile of exceedance is +17.4 dB and the 99th is +19.9 dB.

Exceedance over dialogue reference	Share of audible seconds
≥ 5 dB	47.6%
≥ 10 dB	30.8%
≥ 15 dB	12.6%
≥ 20 dB	1.0%

Table 7.2: Exceedance over the dialogue reference.

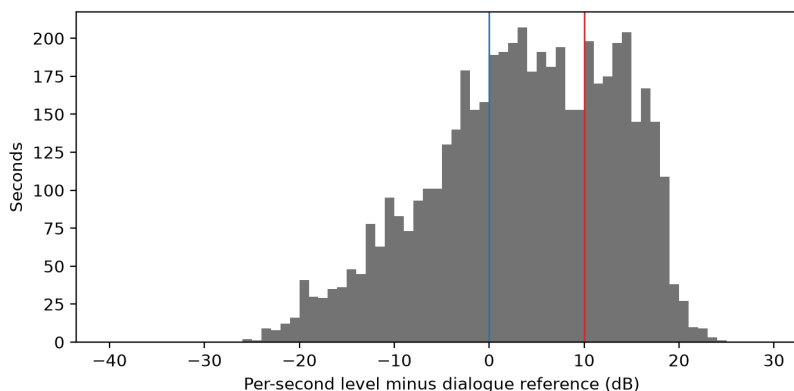


Figure 7.2: Distribution of per-second level relative to the dialogue reference. Blue: the reference. Red: the occupation tolerance $\delta = 10$ dB.

These figures bear on R1. A mix in which dialogue sits comfortably within the range of

the rest of the soundtrack would show little mass beyond +10 dB. The size of that mass, and of the tail beyond +20 dB, measures how far a dialogue-anchored listening level is exceeded and how often. Set beside the loudness range above, they show the difference between the two kinds of description: loudness range reports how widely the level varies, while the exceedance distribution reports how far the level varies *above the one level the listener cares about*.

For this film the answer is specific. Almost a third of the audible running time (30.8%) lies at least 10 dB above the dialogue reference, but only 1.0% lies 20 dB or more above it. The occupied range is a plateau rather than a tail: among occupied seconds the median exceedance is +14.3 dB, the 90th percentile +18.3 dB, and 71.5% fall between +12 and +19 dB, after which the distribution falls away steeply (Figure 7.2). The loudest second in the film is -10.1 LUFS, +24.5 dB above the reference. By peak intensity this is not an extreme soundtrack. What distinguishes it is how much time it spends in one band well above its dialogue. That is the distinction between peak intensity and occupation drawn in Chapter 1, found in the data.¹

7.5.5 Occupation blocks

With tolerance $\delta = 10$ dB, merge gap $g = 5$ s, and minimum length $m = 60$ s, the analysis finds 9 occupation blocks, totalling 16.7 minutes. The longest lasts 4.7 minutes and begins at 1:15:09. Table 7.3 lists them with the implied throttle depth of Chapter 3, the attenuation a listener who had set the volume for dialogue would need to hold each block’s 95th-percentile level at tolerance. Across blocks the median implied throttle is 8.8 dB and the largest is 12.8 dB.

Three blocks fall in the first hour, at about 8, 20, and 36 minutes, each isolated. The other six fall between 1:00:48 and 1:19:53. The last two are separated by only eleven seconds, so that from 1:13:45 to 1:19:53 the soundtrack is effectively a single block of about six minutes. Blocks do not contain all occupied time: of 1598 occupied seconds, 926 (57.9%) fall in blocks and the rest in shorter episodes.

Start	End	Min.	Median E	$Q_{95}(E)$	Throttle (dB)	Subtitled s
0:08:18	0:09:30	1.2	+13.8	+17.0	7.0	0
0:20:11	0:21:13	1.0	+14.0	+18.2	8.2	14
0:36:23	0:38:14	1.9	+14.2	+17.5	7.5	3
1:00:48	1:02:29	1.7	+16.2	+19.1	9.1	10
1:04:24	1:05:56	1.5	+16.8	+20.7	10.7	0
1:06:46	1:08:05	1.3	+16.9	+22.8	12.8	10
1:09:24	1:11:34	2.2	+12.5	+16.5	6.5	48
1:13:45	1:14:58	1.2	+15.5	+18.8	8.8	13
1:15:09	1:19:53	4.7	+16.0	+20.4	10.4	30

Table 7.3: Level-defined occupation blocks of at least 60 s. The last column counts subtitled dialogue seconds inside each block.

An independent, transient-defined count comes from merging version 3.1 barrage regions across gaps of up to 5 s. That procedure yields 0 blocks of at least 60 s, the longest lasting 0 minutes. Figure 7.4 compares the two definitions on a common time axis. Agreement between

¹A plateau with a steep upper edge is what one would expect if loud material were held under a ceiling by compression or limiting in the mix or master. That is a hypothesis about production, not a measurement; the file shows the shape, not how it was made.

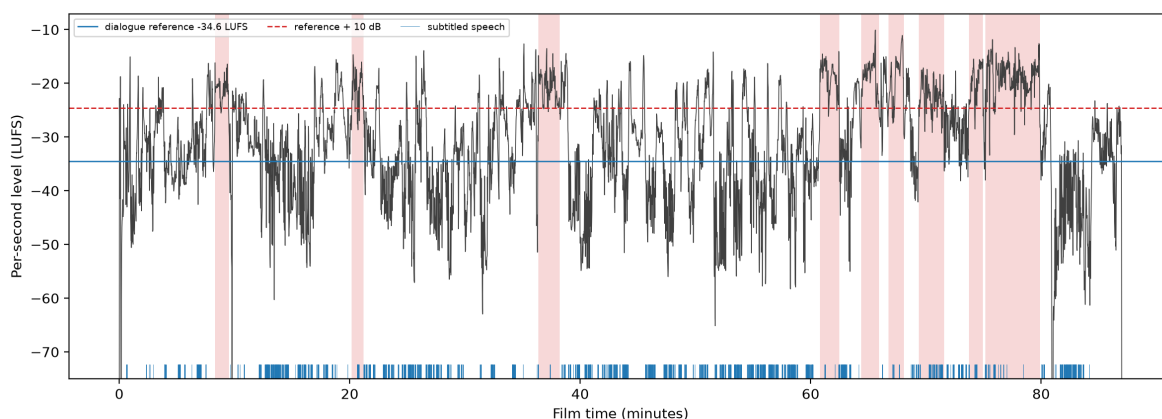


Figure 7.3: Per-second level with the dialogue reference, the occupation tolerance, and occupation blocks shaded. Ticks along the bottom mark subtitled dialogue seconds.

the two definitions would have been evidence that the blocks are properties of the soundtrack rather than of either detector. They do not agree. No merged barrage run reaches 60 s; transient-defined episodes are scattered through the film and short. Inside level-defined blocks the mean barrage score is higher than outside (0.29 against 0.12) and the mean low-frequency onset score much higher (0.60 against 0.04), but neither forms continuous runs above the detectors' thresholds. The blocks are sustained in level; their onset density is episodic within them. Part of the disagreement is built into the barrage detector, whose percentile threshold and three-second minimum fragment long episodes. The rest is plausibly real: bursts of gunfire or impacts over a continuous loud bed of music and effects would produce exactly this pattern. The audition clips can decide between these readings.

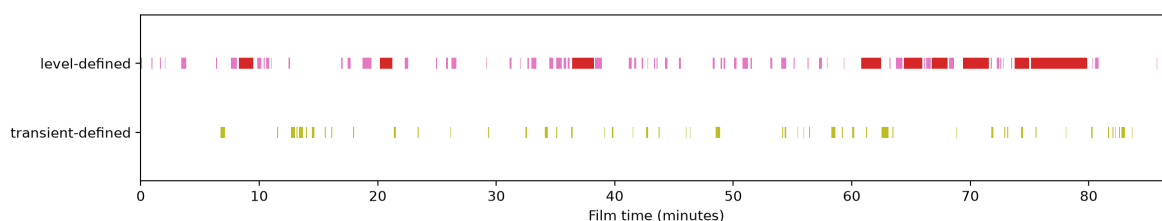


Figure 7.4: Level-defined (top) and transient-defined (bottom) blocks. Dark segments meet the minimum length.

7.5.6 Two regimes

The distribution of blocks and gaps suggests that the film changes character in its last half hour. Table 7.4 and Figure 7.5 make this explicit with fixed ten-minute segments, which involve no choice of boundary. For the first hour, between 13% and 40% of each segment lies 10 dB or more above dialogue and between 30% and 51% at or below it. In the two segments from 1:00:00 to 1:20:00 the proportions invert: 62% and 71% above, 14% and 3% at or below. The last segment, containing the closing dialogue scene and the credits, returns to the pattern of

the first hour.

Segment	$\geq +\delta$ (%)	$\leq D$ (%)	In blocks (%)	Subtitled (%)	Median E
0:00:00–0:10:00	22	35	12	12	+3.3
0:10:00–0:20:00	17	31	0	36	+3.1
0:20:00–0:30:00	18	49	10	38	+0.2
0:30:00–0:40:00	40	30	19	16	+7.7
0:40:00–0:50:00	13	45	0	39	+1.6
0:50:00–1:00:00	18	51	0	51	-0.3
1:00:00–1:10:00	62	14	51	17	+12.5
1:10:00–1:20:00	71	3	75	23	+13.4
1:20:00–1:27:02	6	55	0	29	-0.6

Table 7.4: Ten-minute segments: share of audible seconds at least δ above the dialogue reference, at or below it, inside occupation blocks, and carrying subtitled speech, with median exceedance in dB.

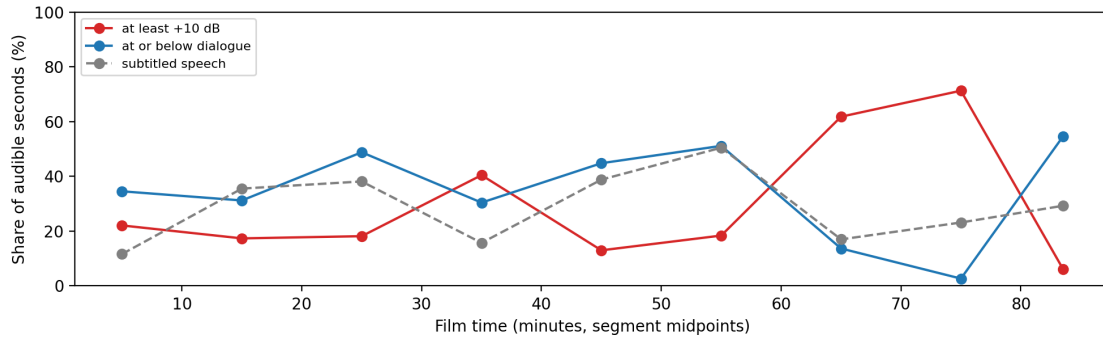


Figure 7.5: Share of audible seconds at least δ above the dialogue reference, at or below it, and carrying subtitled speech, by ten-minute segment.

The span defined by the longest recovery gaps (intervals without a qualifying recovery window) and the short stretches between them runs from 1:00:39 to 1:20:51, 20.2 minutes. Within it, 67.4% of seconds lie at least δ above dialogue, 6.6% at or below it, the median exceedance is +13.1 dB, and 62.6% of seconds belong to occupation blocks. The span contains 3 qualifying recovery windows, totalling 29 seconds; counting every run at or below dialogue level, including runs shorter than five seconds, gives 80 seconds in 32 runs, the longest lasting 13 seconds. Before 1:00:39, the corresponding figures are 21.3% above, 40.4% at or below, and 6.8% in blocks.

These results bear on R2 and R3. The framework predicts that fully turning the volume down for a minute or more occurs in blocks that are both long and deep, that is, with large implied throttle. The report is consistent with the framework if the film contains blocks of that kind. It would count against the framework's account of the report if the film contained no such blocks: if, for example, exceedances were large but brief, or long but shallow.

7.5.7 Dialogue exposed to throttling

Occupation blocks are not free of speech. Across all blocks, 128 subtitled dialogue seconds fall inside them, 8.5% of all subtitled dialogue in the film; the block from 1:09:24 to 1:11:34 alone contains 48. The median level of these seconds is +12.0 dB relative to the reference, against -0.9 dB for subtitled seconds outside blocks. Because the per-second level is that of the whole mix, this does not say the speech is loud; it says the speech occurs inside the loud material. Whether it remains intelligible there is a question of masking, which this analysis does not measure. This is the dialogue a listener loses by turning the volume down or off for the duration of a block, and it quantifies the dilemma of Chapter 1: in these seconds the soundtrack asks to be heard and to be turned down at the same time. The subtitles also give the listener a partial remedy, which is to read what cannot be heard; that the remedy exists at all is a measure of the problem.

7.5.8 Recovery

Using the absolute definition (at least five seconds at or below the dialogue reference, or inaudible), the film contains 105 recovery windows, covering 29.2% of its running time. The longest gap between them lasts 11.5 minutes and begins at 1:09:21. The maximum recovery debt accumulated within a single gap is 2387 dB-s. The longest relative-recovery gap from Table 7.1 lasts 11.6 minutes and begins at 1:09:15. The two definitions, one based on the film's own quietest quarter and the other on its dialogue level, place their longest gaps within six seconds of each other. The location of the film's longest stretch without rest does not depend on how rest is defined. A recovery gap is not an occupation block: a gap measures how long the soundtrack goes without returning to dialogue level, while a block measures how long it stays at least δ above it. A long gap may contain a block, several short ones, or none.

Gap start	Gap end	Minutes
1:09:21	1:20:51	11.5
1:03:27	1:09:00	5.5
0:17:53	0:21:18	3.4
0:36:19	0:39:00	2.7
1:00:39	1:03:18	2.6

Table 7.5: The five longest gaps between absolute recovery windows.

7.5.9 Swells

The swell detector found 78 sustained rises, of which 19 rise by 20 dB or more. Only 4 begin below -55 LUFS, near the audibility floor, where a large rise is more likely to be an entry from near silence or a scene transition than a crescendo. The concern raised in Chapter 5, that large detected rises might mostly be entries from silence, therefore does not hold for this film: most begin from audible levels. 40 of the 78 end in the occupied range, at or above $D + \delta$. These are the swells that matter for the listener model, rises that carry the soundtrack from a level set for dialogue into the range where the volume must come down. Whether they are musical is a question for the audition clips.

7.6 Assessment

R1 is borne out by the file. Measured against its own subtitled dialogue, the film spends 30.8% of its audible running time at least 10 dB above dialogue level and 12.6% at least 15 dB above it, while its dialogue is strongly centred and its loud material is held on a plateau 12 to 19 dB above the reference. Whether this makes the dialogue “mixed too low” or the effects “mixed too high” is the same measurement described two ways.

R2 is borne out in level and not in onset density. The film contains 9 blocks of a minute or more at least 10 dB above dialogue, six of them in its last half hour and one effectively six minutes long. The transient detectors do not find sustained barrages of comparable length; onset activity is elevated but episodic within the blocks.

R3 cannot be tested against the file, but the conditions the framework associates with turning the volume fully down are present: long blocks with implied throttle depths from 6.5 to 12.8 dB (median 8.8 dB), and a span of 20.2 minutes that contains only 29 seconds of qualifying recovery (3 windows). The reported behaviour, turning the volume off for one to two minutes or more, matches the lengths of the blocks in that span.

R4 remains untested.

The case therefore supports the central distinction of the book at the grade “measured”: this soundtrack is not distinguished by extreme peaks but by the amount and continuity of time it spends in one band well above its dialogue, concentrated in a single twenty-minute stretch with almost no return to rest.

Chapter 8

Overdetermination

Level is not the only way a soundtrack claims attention. An abrupt onset claims it; so does a rhythmic low-frequency pulse, a rising swell, and bright, noisy spectral content. This chapter considers what happens when several of these channels are used at once, and what the present measurements can and cannot say about it.

8.1 The idea

A single moment in an action sequence may contain a physical impact, a vocal cry, a percussive hit in the score, a rising orchestral or synthetic swell, and the broadband noise of breaking glass or gunfire. Each layer, taken alone, is ordinary craft. Murch’s account of density describes how layers are distributed so that each remains legible (Murch, 2005), and Chion’s notion of added value describes how sound directs the interpretation of the image (Chion, 1994). The concern here is different. When several channels are elevated together, they may be communicating the same thing, an instruction to attend now, through redundant means. The word *overdetermination* is used for this: an effect produced by more causes than are needed to produce it.

Redundancy is not in itself a defect; it makes a signal robust. The hypothesis of Chapter 3 is narrower. It is that, at a given level of exceedance, a moment in which more salience channels are elevated is more likely to provoke a volume reduction, and that sustained overdetermination contributes to the listener’s sense that the soundtrack never stops demanding attention.

8.2 Measurement

The overdetermination index $\Omega(t)$ counts, for each second, how many of five channels are elevated: level exceedance ($E \geq \delta$), transient barrage, low-frequency onset activity, harshness, and an active swell event. Channels other than exceedance count as elevated when their per-second score exceeds 1.5 on the pipeline’s standardised scale.

Because several channels share components with level (see below), the index is also computed in a residualised form, Ω_{res} . For each channel, a quadratic in $L(t)$ is fitted across audible seconds, and the channel counts as elevated only if it exceeds the raw threshold *and* at least half that margin remains after the level-predicted value is subtracted. By construction $\Omega_{\text{res}} \leq \Omega$.

	Ω	Ω_{res}
Mean within occupation blocks	1.64	1.31
Mean outside occupation blocks	0.84	0.62
Share of audible seconds with ≥ 3 channels	4.5%	2.9%
Share of block seconds with ≥ 3 channels	7.2%	3.6%

Table 8.1: Overdetermination, raw and residualised on level.

The difference between the two columns estimates how much of the apparent co-elevation is mechanical. What remains in the residualised column is co-elevation that level does not explain.

8.3 Result for the case

At first sight the index doubles inside occupation blocks, raw and residualised alike. But one of its five channels is exceedance itself, which is present in nearly every block second by definition. Removing that term leaves the number of *other* channels elevated per second: 0.71 inside blocks against 0.68 outside on the raw scale, and 0.39 against 0.46 after residualising on level. The doubling is carried entirely by exceedance. Excluding it, the blocks are no more overdetermined than the rest of the film, and after removal of the level dependence slightly less so. Seconds with three or more channels elevated are rare everywhere: 2.9% of audible seconds and 3.6% of block seconds on the residualised index.

The channel means tell the same story with one qualification. Low-frequency onset activity is much higher on average inside blocks (0.60 against 0.04), and barrage somewhat higher (0.29 against 0.12), while harshness is about the same (4.49 against 4.26). These elevations are mostly below the channel threshold. The blocks carry more low-frequency rhythmic activity, but not in a way that stacks with the other channels.

For this film, at these thresholds, the soundtrack-side premise of the overdetermination hypothesis is not supported. What distinguishes its occupied stretches is sustained level, with somewhat denser low-frequency activity, not a pile-up of independent salience channels.

8.4 What the index cannot show yet

Three limitations prevent these numbers from testing the hypothesis.

First, the channels are not independent by construction. The barrage and harshness scores each include a level term, and every onset-based measure is audibility-weighted by level. A rise in level therefore raises several channels mechanically. The residualised index addresses the part of this coupling that a smooth function of level can capture.¹

Second, the channels are acoustic, not semantic. Elevated low-frequency onset activity may be a drum, an engine, or a sequence of impacts. The overdetermination hypothesis is about distinct sources carrying the same message, which requires knowing what the sources are. The

¹Residualising on a quadratic in level removes the average dependence of each channel on level, not every interaction. A channel whose relation to level differs between scenes (for example, harshness that rises with level in gunfire but not in music) is only partly corrected.

audition clips exist for this purpose, and Chapter 9 describes how their annotation should be conducted.

Third, the hypothesis is about behaviour. Co-elevation is only interesting if it predicts volume reductions beyond what exceedance predicts, and that requires the response data specified in the next chapter.

8.5 A note on the earlier reading

Version 3's plots appeared to show every channel rising together between about 81 and 84 minutes, and an early reading took this as the clearest instance of overdetermination in the film. Chapter 5 showed that this stretch was quiet and that its apparent intensity was an artefact of level-invariant features. The episode is a warning specific to this chapter's subject: an index that counts coincident elevations will find the most coincidences wherever a shared artefact touches every channel at once. Audibility weighting removes the artefact that produced that particular coincidence. Residualising on level removes much of the mechanical coupling that remains.

8.6 Status

Overdetermination is graded as hypothesised. For this film the index finds no excess co-elevation in occupation blocks once exceedance itself is set aside. That result is limited by the acoustic channels chosen and their thresholds, and it says nothing yet about sources or behaviour: a scream, a gunshot, and a drum hit in the same second could all register in one channel. The hypothesis is therefore weakened rather than rejected. Source labels from the blinded annotation are the next test of it.

Part IV

Response and Intervention

Chapter 9

The Hand on the Volume

The framework’s hypotheses are about behaviour: when listeners reduce the volume, how far, and for how long. This chapter specifies how that behaviour can be recorded and how a study should be designed so that the resulting data can confirm or refute the framework rather than decorate it.

9.1 Origin of the measure

The idea of treating volume behaviour as data predates this analysis. A set of design notes from July 2025, kept in the author’s public repository, proposed a tool provisionally called MuteNet: a background process that records when viewers mute, when they drop the volume sharply, and when they restore it after a loud scene, and that aggregates these records across viewers to identify consensus moments of unpleasant sound and to train models of them (Flyxion, 2025). The same notes proposed the intervention developed in Chapter 10. The present chapter turns the first proposal into a measurement protocol.

9.2 Why volume behaviour

Volume behaviour has three advantages as a dependent variable. It is produced during viewing rather than recalled afterwards. It is continuous in time and can be aligned exactly with the soundtrack. And, as the noise-control literature suggests (Glass and Singer, 1972), it is a direct expression of the listener exercising control over the sound, which is the phenomenon at issue. Its disadvantage is that it confounds several motives (level, distortion, content, fatigue, and the wish to hear the next line of dialogue), which is why it must be combined with the chain measurements of Chapter 6 and the annotations described below.

9.3 Recording

The script `throttle-log.lua` (Appendix C) runs inside the `mpv` player and writes a record each time the volume, mute state, or pause state changes, stamped with the film time, the wall-clock time, the slider value, and the corresponding gain in decibels. A key binding records a marker when something bothers the viewer without a volume change. The protocol requires

that system volume and amplifier settings be fixed and recorded before viewing and that the viewer adjust level only within the player, so that the logged gain is the only gain that changes.

From the log the following variables are derived:

Reduction onset. A time at which gain decreases by at least 3 dB within two seconds.

Reduction depth. The gain change in dB, with full muting or a slider at zero treated as a separate outcome, *off*.

Listener recovery time. The time from a reduction until gain returns to within 3 dB of its prior value.

Marker. A timestamp without gain change.

Listener recovery time is the behavioural counterpart of the soundtrack’s recovery windows. The comparison of the two is the most direct test the framework admits: whether listeners restore the volume when, and only when, the soundtrack returns to dialogue level.

9.4 Annotation of candidates

The audition clips produced by the pipeline allow the acoustic detectors to be tied to identified sources, but only under conditions that prevent the annotator’s expectations from determining the labels.

1. **Blind presentation.** Clips are renamed with random identifiers and shuffled across categories. Detector name, score, and timestamp are withheld until labelling is complete.
2. **Random controls.** Thirty to fifty windows drawn uniformly from the audible portion of the film are mixed in with the detector clips. Without them, the proportion of detector clips labelled as unpleasant has no baseline and cannot show that the detectors select anything.
3. **Several dimensions.** Each clip receives a source label from a fixed vocabulary (Appendix B), a role (voice, music, effect, ambience, mixed), an intensity rating from one to five, and a binary judgement of whether the listener would reduce the volume.
4. **More than one listener.** Three to five listeners allow inter-rater agreement to be computed and turn the single-listener limitation into a measurement of variation between listeners.

The pipeline’s annotation summary then reports, for each detector, the share of its clips that were not false positives and the distribution of source labels, which can be compared with the same distribution for the random controls.

9.5 Design

9.5.1 Pre-registration

Before any response data are examined, the predictions of Chapter 3 are to be written down with their direction and the variables that operationalise them. The listening report was

produced before the analysis. The analysis has since been seen by the listener, and any further viewing of this film by the same listener is a repeat viewing informed by the plots. That contamination is unavoidable for this listener and film and must be stated; it is avoidable for new listeners and new films.

9.5.2 Comparison corpus

A single film cannot show that its occupation is unusual. A corpus of five to ten films, including films of the same genre from earlier decades and films of other genres from the same period, analysed with a common dialogue-referenced method and thresholds fixed across the corpus, would allow the case's exceedance and block statistics to be placed in a distribution. The percentile-based detectors of the current pipeline would need pooled thresholds for this purpose.

9.5.3 Playback conditions

Each listener should view under at least two conditions: the chain of interest and a neutral reference (headphones or monitors of known response), with order counterbalanced across listeners and films. This separates what listeners respond to in the mix from what they respond to in the chain, which is the objection a reviewer would raise first to any single-chain study.

9.6 Analysis

The primary model is a discrete-time hazard model. For each second in which the listener has not already reduced the volume, the outcome is whether a reduction onset occurs in that second. Predictors are current exceedance $E(t)$; time already elapsed in the current occupation block; recovery debt $K(t)$; the residualised overdetermination index; the playback condition; and listener and film random effects. The hypotheses of Chapter 3 map onto coefficients: occupation over level (block elapsed time and E outperform instantaneous level alone, and whole-film statistics add nothing), recovery debt (positive coefficient on K), overdetermination (positive coefficient on Ω at fixed E), and chain amplification (positive effect of the chain condition, larger at high E).

A secondary model predicts listener recovery time from the time until the soundtrack's next absolute recovery window. A tertiary analysis predicts the *off* outcome specifically from block length and implied throttle depth.

9.7 Aggregation

The MuteNet proposal envisaged aggregation across many viewers. The log format above is sufficient for that purpose and contains nothing about the viewer beyond their volume behaviour and the film's identity. Aggregated over viewers, reduction onsets form an empirical annoyance curve for a film, which could be published alongside it much as loudness metadata already is, and which would give sound designers a direct view of where home listeners reach for the control.

Chapter 10

Remixing for the Listener

A listener who reduces the volume during an action sequence is already remixing the film, crudely, with a single global gain. This chapter considers how that remix can be done less crudely, beginning with what is possible now on any 5.1 source and ending with more speculative tools.

10.1 A continuum of interventions

The July 2025 notes that proposed the response measure of Chapter 9 also proposed a pair of tools, a “whisperizer” and a “mufflerizer” (Flyxion, 2025). In their most radical form they would re-render all dialogue as whispered speech, detect gunshots, explosions, and screams, remove them, replace each class with a stylised musical substitute, and signal the substitution visually on screen. Taken literally this is a transformation of the film into a different work. Taken as the far end of a continuum, it clarifies the design space:

1. **Global gain.** The listener’s volume control. Available everywhere; sacrifices dialogue to control action or the reverse.
2. **Global dynamic range control.** Consumer “night mode” and decoder compression profiles. Reduces the gap between loud and quiet without regard to source.
3. **Channel-based rebalancing.** Raising the channel that carries most dialogue relative to the others, then compressing. Available on any discrete multichannel source.
4. **Separated rebalancing.** Estimating a dialogue stem by source separation, for sources in which dialogue is not isolated by channel, and rebalancing it against the rest.
5. **Class-specific attenuation.** Detecting particular event classes and attenuating them selectively.
6. **Class-specific substitution.** Replacing detected classes with other sounds and, optionally, visual cues.

10.2 A dialogue-forward downmix

Level 3 is available today with `ffmpeg` or with the `mpv` player, which accepts `ffmpeg` filters during playback. The following plays a 5.1 film with the centre channel forward, everything else reduced, the result compressed and limited, and the output in mono:

```
mpv --af="lavfi=[pan=mono|c0=1.0*FC+0.35*FL+0.35*FR+0.25*SL+0.25*SR,acompressor=threshold=-24dB:ratio=6:
attack=5:release=250:makeup=6dB,alimiter=limit=-3dB]" film.mkv
```

The same filter chain in an `ffmpeg` command renders a permanently rebalanced copy of the file:

```
ffmpeg -i film.mkv -map 0:v -map 0:a:0 -c:v copy \
  -af "pan=mono|c0=1.0*FC+0.35*FL+0.35*FR+0.25*SL+0.25*SR,acompressor=threshold=-24dB:ratio=6:attack=5:
release=250:makeup=6dB,alimiter=limit=-3dB" \
  -c:a aac -b:a 160k film-dialogue-forward.mkv
```

Each stage has a purpose. The `pan` stage reweights channels before they are summed, raising dialogue relative to effects and music by roughly 9 to 12 dB compared with an equal-weight sum, and produces a single channel so that a mono chain receives the whole mix regardless of which conductor its plug connects. The compressor reduces the level of passages above its threshold by a factor of six, with a fast attack to catch impacts and a moderate release to avoid audible pumping between lines, and restores overall level with make-up gain. The limiter enforces a ceiling so that transients the compressor misses cannot drive a downstream preamplifier into clipping. The weights and time constants are starting points, tested for correct operation on a synthetic multichannel file, and are meant to be adjusted by ear.

10.3 Limitations

Channel-based rebalancing inherits the limits of the dialogue proxy of Chapter 3. Effects panned to the centre are raised with the dialogue; dialogue spread into the front pair is partly lowered with the effects. Compression changes the character of the mix everywhere, not only where it is needed. And on stereo sources there is no centre channel to raise, which is where level 4 becomes necessary.

10.4 Beyond channels

Levels 4 to 6 depend on models. Neural source separation can estimate a dialogue stem from a stereo mix, and sound event detection models trained on large labelled corpora (Gemmeke et al., 2017) can mark gunshots, explosions, and screams in time. Next-generation broadcast audio formats already carry dialogue as a separate object with user-adjustable level, which makes level 4 a matter of metadata rather than estimation for programmes delivered in those formats. Class-specific substitution (level 6) is a creative transformation rather than a correction; it has an accessibility rationale for viewers with sound sensitivity, for whom certain classes of sound are the barrier to watching at all, and a clear risk of misclassification, which is a reason to signal substitutions visibly rather than silently.

10.5 The objection from intent

An intervention of any of these kinds alters what the filmmakers made. The objection deserves a direct answer. The dynamic relationships of a film mix are authored for a calibrated room (Chapter 2). In an uncalibrated home, through an arbitrary chain, those relationships are already not reproduced, and the listener already intervenes with the volume control, in the crudest possible way and at the cost of whole scenes of dialogue. The choice is not between the authored mix and an altered one. It is between an alteration performed by hand, reactively, with one global control, and an alteration designed to keep the dialogue the film depends on while bringing the rest into a range the room can carry.

10.6 Evaluation

Hypothesis 5 of Chapter 3 states what the intervention must achieve: fewer and shorter volume reductions without loss of dialogue comprehension. The design of Chapter 9 tests it directly by adding the rebalanced mix as a playback condition, with comprehension assessed by brief questions on dialogue content after each viewing. An intervention that reduces throttling by making dialogue as hard to follow as the action is not a success.

Part V

Conclusion

Chapter 11

What Has Been Shown

11.1 The argument in brief

The difficulty of watching certain films at home is usually described as a matter of loudness. This monograph has argued that the better description is occupation: the extent to which a soundtrack stays far above the level its dialogue requires, and for how long it does so without returning. Standard loudness measures summarise distributions and cannot see this temporal property. A listener anchored to dialogue meets it directly, as a sequence of exceedances to be managed by hand, and a long, deep block of exceedance leaves only one practical response, which is to turn the sound off and wait.

11.2 Graded claims

Table 11.1 collects the principal claims of the book with their grades.

Claim	Grade	Chapter
Integrated loudness and loudness range are distributional and do not describe the temporal arrangement of exceedances	Established	2
Cinema reproduction is calibrated to a reference level; home reproduction generally is not	Established	2
Perceived control over noise reduces its aftereffects	Established	2
Speech intelligibility depends most on roughly the one to four kilohertz region	Established	2
The default <code>ffmpeg</code> mono downmix of a tagged 5.1 stream omits the LFE channel	Measured	4
The film's relative recovery windows are ordinarily frequent (median gap 17.2 s) but absent for one stretch of 694.7 s	Measured	7
The film's level is high and sustained from roughly 61 to 80 minutes	Measured	7
Low-frequency onset activity recurs throughout the film rather than only at its climax	Measured	7

Claim	Grade	Chapter
The film's integrated loudness is -21.6 LUFS and its loudness range 22.3 LU	Measured	7
The dialogue reference lies 13.0 dB below integrated loudness	Measured	7
30.8% of audible time lies at least 10 dB above subtitled dialogue level, but only 1.0% at least 20 dB above; loud material forms a plateau from about +12 to +19 dB	Measured	7
The film contains 9 occupation blocks of at least a minute, six of them after the first hour	Measured	7
From 1:00:39 to 1:20:51 the soundtrack contains only 29 s of qualifying recovery	Measured	7
Relative and absolute definitions of recovery locate the longest gap in the same place	Measured	7
The loud plateau reflects compression or limiting in the mix or master	Hypothesised	7
8.5% of subtitled dialogue falls inside occupation blocks	Measured	7
The centre-channel proxy locates dialogue poorly (precision and recall near one half) but estimates its level within about 1 dB	Measured	7
Excluding exceedance itself, occupation blocks are not more overdetermined than the rest of the film	Measured	8
Most large swells begin from audible levels rather than near silence	Measured	7
Level-defined occupation blocks coincide with sustained transient barrages	Rejected	7
Volume reductions are predicted by occupation variables better than by level or whole-film statistics (H1)	Hypothesised	3
Reduction probability rises with recovery debt (H2)	Hypothesised	3
Reduction probability rises with overdetermination at fixed exceedance (H3)	Hypothesised (premise weakened)	8
The bass-amplifier chain widened the dialogue-to-action gap and added distortion to loud passages (H4)	Hypothesised	6
A dialogue-forward downmix reduces throttling without loss of comprehension (H5)	Hypothesised	10
Version 3 swell candidates identify swells	Rejected	5
Version 3 harshness peaks identify harsh material	Rejected	5
The stretch from 81 to 84 minutes is the most intense in the film (it is the quiet closing dialogue scene)	Rejected	5
Percentile-defined totals show that the film's recovery is scarce	Rejected	5
The early analysis had established that the film "has plenty of dynamic range"	Rejected	5
Swell events of 30 dB or more are musical crescendos	Rejected (untested)	5
The low-frequency detector identifies drums	Rejected	5

Claim	Grade	Chapter
-------	-------	---------

Table 11.1: Principal claims and their grades.

11.3 Limitations

The study rests on one film, one listener, one viewing, and one unmeasured reproduction chain. Its dialogue reference depends on subtitle timing, which marks when speech occurs but not every utterance, and which may be imperfectly aligned with the analysed release. Its frame-level detectors, apart from the swell detector, still use unweighted features and film-internal percentile thresholds, even though the occupation measures now use BS.1770 loudness. Its residualised overdetermination index removes only the smooth dependence of each channel on level. Its response data consist of a retrospective report without timestamps. Each of these limitations has a remedy specified in the text: pooled thresholds across a corpus, psychoacoustic replacements for the spectral descriptors, source labels from blinded annotation, the logging protocol of Chapter 9, and the chain measurements of Chapter 6.

11.4 A programme

The research programme that follows from the framework has five components, in order of cost:

1. characterise the reproduction chain with a sine sweep and a level calibration;
2. conduct blind, multi-listener annotation of the audition clips with random controls;
3. log volume behaviour for new viewings under at least two playback conditions, including the dialogue-forward downmix;
4. analyse a comparison corpus with common, dialogue-referenced thresholds;
5. fit the hazard models of Chapter 9 and report which hypotheses survive.

11.5 Closing

The instruments built for this study failed several times before they measured anything, and the failures were more convincing than the successes until they were checked. That history is part of the result. A complaint about a soundtrack is easy to confirm with plots that are drawn after the complaint is made. It is harder, and more useful, to specify in advance what the soundtrack would have to contain for the complaint to be explained, to measure whether it does, and to name the listener data that could show the explanation wrong. The concept of occupation is offered as the core of such a specification.

Appendix A

Reproducing the Results

The numerical results of Chapter 7 are produced in two steps from the film file. The first step decodes the film and takes the longest; the second runs in seconds.

```
# 1. Frame features, detector events, audition clips
python3 code/analyze-soundtrack-v3.1.py drive-through-fire-2026.mkv \
    --title "Drive Through Fire" --out sound-analysis-v3.1

# (or, reusing an existing version 3 frame table)
python3 code/analyze-soundtrack-v3.1.py drive-through-fire-2026.mkv \
    --from-frames sound-analysis-v3/frames.csv --out sound-analysis-v3.1

# 2. Dialogue-referenced measures, figures, and LaTeX macros
python3 code/occupation.py \
    --frames sound-analysis-v3.1/frames.csv \
    --events sound-analysis-v3.1/events.csv \
    --film drive-through-fire-2026.mkv \
    --subtitles drive-through-fire-2026.srt \
    --out results

# supplementary statistics and the segment figure
python3 code/occupation-extras.py --results results

# 3. Blinded annotation set with random controls (Chapter 9)
python3 code/blind-audition.py drive-through-fire-2026.mkv \
    --audition sound-analysis-v3.1/audition \
    --frames sound-analysis-v3.1/frames.csv --out blind --controls 40

# 4. Build the manuscript
pdflatex main.tex && pdflatex main.tex
```

Step 2 decodes the film once more to measure BS.1770 loudness (cached in `results/loudness.csv` for later runs), estimates the subtitle offset, and writes `results/results.tex`, which the manuscript reads at compile time, together with `fig-occupation.png`, `fig-exceedance.png`, `fig-blocks.png`, a per-second table `seconds.csv`, and `occupation.json`. If `results/results.tex` is absent, every dependent value prints as **[pending]**.

Dependencies are `ffmpeg`, Python 3 with `numpy`, `scipy`, `pandas`, `matplotlib`, `soundfile`, and `librosa`, and a standard L^AT_EX installation.

Appendix B

Annotation Instrument

B.1 Label vocabulary

Source labels, of which one or more may be assigned to a clip, joined with +:

DRUM	MUSIC_SWELL	MUSIC_OTHER	SIREN
ALARM	GUNSHOT	EXPLOSION	PUNCH
CRASH	GLASS	ENGINE	TIRES
YELLING	SCREAM	CROWD	DIALOGUE
FALSE_POSITIVE	UNSURE		

B.2 Fields

Field	Content
<code>clip</code>	Random identifier assigned at blinding
<code>label</code>	Source labels from the vocabulary
<code>role</code>	voice, music, effect, ambience, or mixed
<code>intensity</code>	1 (barely noticeable) to 5 (overwhelming)
<code>would_reduce</code>	yes or no: would the listener reduce the volume
<code>notes</code>	Free text

Detector category, score, film time, and whether the clip is a random control are held in a separate key file (`key.csv`, written by `blind-audition.py`) and joined to the labelled sheet on the `clip` column only after labelling is complete. Random control clips appear in the key with category `RANDOM_CONTROL`.

Appendix C

Code

The four programs used in this study are reproduced in full. All are released into the public domain.

C.1 Feature extraction and detection

```
1  #!/usr/bin/env python3
2
3  """
4  Soundtrack Structural Analysis v3.1
5  =====
6
7  Changes from v3
8  -----
9
10 1. Level gating.
11 Onset strength, spectral centroid, flatness and zero-crossing rate
12 are all (nearly) level-invariant. In v3 a click at -65 dBFS scored
13 the same as a crash at -25 dBFS, and digital silence / dither
14 produced the largest HARSH values in the film. Every shape or
15 transient feature is now multiplied by an audibility weight
16 derived from absolute level, and robust statistics are computed
17 over audible frames only.
18
19 2. SWELL is an event detector, not a frame trace.
20 In v3, more than half the positive-gradient frames were exactly 0,
21 so the MAD was 0, robust_z() returned all zeros, and the swell
22 score reduced to -0.25 * onset+. The swell plot was an inverted
23 transient plot. v3.1 measures  $S(t, D) = L(t + D) - L(t)$  for several
24 windows D over 0.25 s energy bins, and requires the rise to be
25 sustained (mostly monotonic, not carried by one step).
26
27 3. Audition clips + annotation CSV.
28 Top-N candidates per category are cut from the source with ffmpeg
29 into audition/<category>/, with an .m3u playlist per category and
30 a blank annotations.csv to fill in.
31
32 4. --from-frames reuses an existing frames.csv (v3 or v3.1), so the
33 post-processing can be rerun without re-decoding the film.
34
35 5. --annotations reads a filled-in annotations.csv and reports
36 per-category precision plus a labelled timeline.
37
38 These are still acoustic classifications, not semantic recognition.
39 """
```

```

40
41 import argparse
42 import shutil
43 import subprocess
44 import tempfile
45 from pathlib import Path
46
47 import librosa
48 import matplotlib
49 matplotlib.use("Agg")
50 import matplotlib.pyplot as plt
51 import numpy as np
52 import pandas as pd
53 import soundfile as sf
54
55 from scipy.ndimage import gaussian_filter1d, median_filter
56 from scipy.signal import find_peaks
57
58
59 LABELS = [
60     "DRUM", "MUSIC_SWELL", "MUSIC_OTHER",
61     "SIREN", "ALARM",
62     "GUNSHOT", "EXPLOSION", "PUNCH", "CRASH", "GLASS",
63     "ENGINE", "TIRES",
64     "YELLING", "SCREAM", "CROWD",
65     "DIALOGUE",
66     "FALSE_POSITIVE", "UNSURE",
67 ]
68
69
70 # =====
71 # Utilities
72 # =====
73
74 def timestamp(seconds):
75     seconds = max(0.0, float(seconds))
76     h = int(seconds // 3600)
77     m = int((seconds % 3600) // 60)
78     s = seconds % 60
79     return f"{h:02d}:{m:02d}:{s:05.2f}"
80
81
82 def file_stamp(seconds):
83     seconds = max(0.0, float(seconds))
84     h = int(seconds // 3600)
85     m = int((seconds % 3600) // 60)
86     s = int(seconds % 60)
87     return f"{h:02d}-{m:02d}-{s:02d}"
88
89
90 def robust_z(x, mask=None):
91     """
92     Median/MAD z-score, with statistics taken over `mask` if given.
93
94     Falls back to IQR, then standard deviation, when the MAD is
95     degenerate. (A zero MAD is what silently killed the v3 swell
96     score.)
97     """
98     x = np.asarray(x, dtype=np.float64)
99     ref = x
100     if mask is not None and np.count_nonzero(mask) > 100:
101         ref = x[mask]
102     ref = ref[np.isfinite(ref)]
103     if len(ref) == 0:
104         return np.zeros_like(x)
105

```

```

106     med = np.median(ref)
107     scale = 1.4826 * np.median(np.abs(ref - med))
108
109     if scale < 1e-12:
110         q75, q25 = np.percentile(ref, [75, 25])
111         scale = (q75 - q25) / 1.349
112     if scale < 1e-12:
113         scale = np.std(ref)
114     if scale < 1e-12:
115         return np.zeros_like(x)
116
117     return (x - med) / scale
118
119
120 def normalize01(x, mask=None):
121     x = np.asarray(x, dtype=np.float64)
122     ref = x[mask] if mask is not None and np.count_nonzero(mask) > 100 else x
123     ref = ref[np.isfinite(ref)]
124     if len(ref) == 0:
125         return np.zeros_like(x)
126     lo, hi = np.percentile(ref, [5, 95])
127     if hi <= lo:
128         return np.zeros_like(x)
129     return np.clip((x - lo) / (hi - lo), 0.0, 1.0)
130
131
132 def level_weight(db, floor, full):
133     """0 at or below `floor` dBFS, 1 at or above `full`, linear between."""
134     return np.clip((np.asarray(db) - floor) / (full - floor), 0.0, 1.0)
135
136
137 def run(cmd, quiet=False):
138     if not quiet:
139         print("+", " ".join(str(x) for x in cmd))
140     subprocess.run(cmd, check=True)
141
142
143 def require(program):
144     if shutil.which(program) is None:
145         raise SystemExit(f"Required program not found: {program}")
146
147
148 def contiguous_regions(mask, times, minimum_duration=1.0):
149     mask = np.asarray(mask, dtype=bool)
150     if len(mask) == 0 or not np.any(mask):
151         return []
152
153     changes = np.diff(mask.astype(np.int8))
154     starts = list(np.where(changes == 1)[0] + 1)
155     ends = list(np.where(changes == -1)[0] + 1)
156     if mask[0]:
157         starts.insert(0, 0)
158     if mask[-1]:
159         ends.append(len(mask))
160
161     regions = []
162     for start, end in zip(starts, ends):
163         if end <= start:
164             continue
165         t0 = float(times[start])
166         t1 = float(times[min(end - 1, len(times) - 1)])
167         if t1 - t0 >= minimum_duration:
168             regions.append((start, end, t0, t1))
169     return regions
170
171

```

```

172 # =====
173 # Arguments
174 # =====
175
176 parser = argparse.ArgumentParser(
177     description="Soundtrack structural analysis v3.1"
178 )
179 parser.add_argument("input", help="Movie/audio file (needed for audition clips)")
180 parser.add_argument("--out", default="sound-analysis-v3.1")
181 parser.add_argument("--title", default=None,
182     help="Plot title prefix (default: input file stem)")
183 parser.add_argument("--audio-stream", default="0:a:0",
184     help="ffmpeg stream selector; default: 0:a:0")
185 parser.add_argument("--chunk", type=float, default=60.0)
186 parser.add_argument("--sr", type=int, default=16000)
187 parser.add_argument("--hop", type=int, default=512)
188 parser.add_argument("--keep-stems", action="store_true")
189
190 parser.add_argument("--from-frames", default=None,
191     help="Reuse an existing frames.csv instead of decoding")
192
193 parser.add_argument("--gate-floor", type=float, default=-65.0,
194     help="Broadband dBFS at which features get zero weight")
195 parser.add_argument("--gate-full", type=float, default=-45.0,
196     help="Broadband dBFS at which features get full weight")
197 parser.add_argument("--low-gate-floor", type=float, default=-70.0)
198 parser.add_argument("--low-gate-full", type=float, default=-50.0)
199
200 parser.add_argument("--swell-windows", default="2,4,6,8",
201     help="Comma-separated swell windows in seconds")
202 parser.add_argument("--swell-min-rise", type=float, default=6.0,
203     help="Minimum sustained rise in dB")
204
205 parser.add_argument("--audition", type=int, default=20,
206     help="Clips per category (0 disables)")
207 parser.add_argument("--pre", type=float, default=12.0,
208     help="Seconds before a point event")
209 parser.add_argument("--post", type=float, default=8.0,
210     help="Seconds after a point event")
211
212 parser.add_argument("--annotations", default=None,
213     help="Filled-in annotations.csv to summarize/plot")
214
215 args = parser.parse_args()
216
217 source = Path(args.input)
218 out = Path(args.out)
219 out.mkdir(parents=True, exist_ok=True)
220 title = args.title or source.stem
221
222 require("ffmpeg")
223 require("ffprobe")
224
225
226 # =====
227 # Feature extraction (unchanged from v3 apart from refactoring)
228 # =====
229
230 def probe_channels(path):
231     probe = subprocess.run(
232         ["ffprobe", "-v", "error", "-select_streams", "a:0",
233          "-show_entries", "stream=channels,channel_layout,sample_rate",
234          "-of", "default=noprint_wrappers=1:nokey=0", str(path)],
235         check=True, capture_output=True, text=True,
236     )
237     print("\nSOURCE AUDIO\n-----")

```

```

238     print(probe.stdout.strip(), "\n")
239     for line in probe.stdout.splitlines():
240         if line.startswith("channels="):
241             try:
242                 return int(line.split("=", 1)[1])
243             except ValueError:
244                 pass
245     print("Warning: could not determine channel count.")
246     return None
247
248
249 def extract_frames(path, channels):
250     if args.keep_stems:
251         stem_dir = out / "analysis-stems"
252         stem_dir.mkdir(exist_ok=True)
253         temp = None
254     else:
255         temp = tempfile.TemporaryDirectory()
256         stem_dir = Path(temp.name)
257
258     broadband_path = stem_dir / "broadband.wav"
259     low_path = stem_dir / "low.wav"
260     center_path = stem_dir / "center.wav"
261     noncenter_path = stem_dir / "noncenter.wav"
262
263     base = ["ffmpeg", "-y", "-hide_banner", "-loglevel", "error",
264            "-i", str(path), "-map", args.audio_stream, "-vn"]
265
266     print("Creating lightweight analysis stems...\n")
267     run(base + ["-ac", "1", "-ar", str(args.sr), "-c:a", "pcm_s16le",
268            str(broadband_path)])
269     run(base + ["-af", "lowpass=f=250", "-ac", "1", "-ar", "8000",
270            "-c:a", "pcm_s16le", str(low_path)])
271
272     has_center = channels is not None and channels >= 3
273     if has_center:
274         run(base + ["-af", "pan=mono|c0=FC", "-ar", str(args.sr),
275            "-c:a", "pcm_s16le", str(center_path)])
276         pan = ("pan=mono|c0=0.25*FL+0.25*FR+0.25*SL+0.25*SR"
277            if channels >= 6 else "pan=mono|c0=0.5*FL+0.5*FR")
278         run(base + ["-af", pan, "-ar", str(args.sr),
279            "-c:a", "pcm_s16le", str(noncenter_path)])
280     else:
281         print("\nNo discrete center channel; using broadband mono.")
282         shutil.copyfile(broadband_path, center_path)
283         shutil.copyfile(broadband_path, noncenter_path)
284
285     broad_file = sf.SoundFile(broadband_path)
286     low_file = sf.SoundFile(low_path)
287     center_file = sf.SoundFile(center_path)
288     noncenter_file = sf.SoundFile(noncenter_path)
289
290     broad_sr = broad_file.samplerate
291     low_sr = low_file.samplerate
292     duration = len(broad_file) / broad_sr
293
294     print(f"\nDuration: {timestamp(duration)}\n")
295
296     columns = {k: [] for k in [
297         "time", "loudness_db", "onset", "centroid_hz", "flatness", "zcr",
298         "low_db", "low_onset", "center_db", "noncenter_db"]}
299
300     chunk_start = 0.0
301     chunk_number = 0
302     low_hop = max(1, int(args.hop * low_sr / broad_sr))
303

```

```

304 while chunk_start < duration:
305     chunk_number += 1
306     chunk_duration = min(args.chunk, duration - chunk_start)
307     print(f"\rChunk {chunk_number:03d} {timestamp(chunk_start)}",
308           end="", flush=True)
309
310     for f in (broad_file, center_file, noncenter_file):
311         f.seek(int(chunk_start * broad_sr))
312     low_file.seek(int(chunk_start * low_sr))
313
314     nb = int(chunk_duration * broad_sr)
315     nl = int(chunk_duration * low_sr)
316
317     y = broad_file.read(nb, dtype="float32", always_2d=False)
318     center = center_file.read(nb, dtype="float32", always_2d=False)
319     noncenter = noncenter_file.read(nb, dtype="float32", always_2d=False)
320     low = low_file.read(nl, dtype="float32", always_2d=False)
321
322     if len(y) < 2048:
323         break
324
325     def db_rms(sig, frame, hop):
326         r = librosa.feature.rms(y=sig, frame_length=frame, hop_length=hop)[0]
327         return librosa.amplitude_to_db(np.maximum(r, 1e-10), ref=1.0)
328
329     feats = [
330         db_rms(y, 2048, args.hop),
331         librosa.onset.onset_strength(y=y, sr=broad_sr, hop_length=args.hop),
332         librosa.feature.spectral_centroid(
333             y=y, sr=broad_sr, n_fft=2048, hop_length=args.hop)[0],
334         librosa.feature.spectral_flatness(
335             y=y, n_fft=2048, hop_length=args.hop)[0],
336         librosa.feature.zero_crossing_rate(
337             y, frame_length=2048, hop_length=args.hop)[0],
338         db_rms(low, 1024, low_hop),
339         librosa.onset.onset_strength(y=low, sr=low_sr, hop_length=low_hop),
340         db_rms(center, 2048, args.hop),
341         db_rms(noncenter, 2048, args.hop),
342     ]
343
344     n = min(len(f) for f in feats)
345     if n == 0:
346         chunk_start += chunk_duration
347         continue
348
349     # Drop the final frame of each chunk: with centred framing it
350     # sits exactly on the next chunk's first frame.
351     if chunk_start + chunk_duration < duration and n > 1:
352         n -= 1
353
354     t = chunk_start + librosa.frames_to_time(
355         np.arange(n), sr=broad_sr, hop_length=args.hop)
356
357     columns["time"].append(t)
358     for key, f in zip(list(columns)[1:], feats):
359         columns[key].append(np.asarray(f[:n], dtype=np.float64))
360
361     chunk_start += chunk_duration
362
363 print("\n")
364
365 for f in (broad_file, low_file, center_file, noncenter_file):
366     f.close()
367
368 if args.keep_stems:
369     print("Analysis stems retained in:", stem_dir)

```

```

370     else:
371         temp.cleanup()
372
373     return pd.DataFrame({k: np.concatenate(v) for k, v in columns.items()}), duration
374
375 channels = probe_channels(source)
376
377 if args.from_frames:
378     raw_cols = ["time", "loudness_db", "onset", "centroid_hz", "flatness",
379                 "zcr", "low_db", "low_onset", "center_db", "noncenter_db"]
380     df = pd.read_csv(args.from_frames, usecols=raw_cols)
381     df = df.drop_duplicates("time").sort_values("time").reset_index(drop=True)
382     duration = float(df["time"].iloc[-1])
383     print(f"Reusing {args.from_frames}: {len(df)} frames, "
384           f"{timestamp(duration)}\n")
385 else:
386     df, duration = extract_frames(source, channels)
387
388 if len(df) == 0:
389     raise SystemExit("No analysis frames generated.")
390
391 times = df["time"].to_numpy()
392 loud_db = df["loudness_db"].to_numpy()
393 onset = df["onset"].to_numpy()
394 centroid = df["centroid_hz"].to_numpy()
395 flatness = df["flatness"].to_numpy()
396 zcr = df["zcr"].to_numpy()
397 low_db = df["low_db"].to_numpy()
398 low_onset = df["low_onset"].to_numpy()
399 center_db = df["center_db"].to_numpy()
400 noncenter_db = df["noncenter_db"].to_numpy()
401
402 frame_rate = 1.0 / float(np.median(np.diff(times)))
403
404
405 # =====
406 # Audibility gating
407 # =====
408
409 w = level_weight(loud_db, args.gate_floor, args.gate_full)
410 w_low = level_weight(low_db, args.low_gate_floor, args.low_gate_full)
411
412 audible = w > 0.5
413 audible_low = w_low > 0.5
414
415 silent_fraction = 1.0 - np.mean(w > 0)
416 print(f"Frames below gate floor ({args.gate_floor:.0f} dBFS): "
417       f"{100 * silent_fraction:.1f}%")
418
419 # Statistics are taken over audible frames, then the result is
420 # multiplied by the audibility weight so quiet material cannot score
421 # as harsh or percussive merely because its spectrum is flat.
422
423 z_loud = robust_z(loud_db, audible)
424 z_onset = w * robust_z(onset, audible)
425 z_centroid = w * robust_z(centroid, audible)
426 z_flatness = w * robust_z(flatness, audible)
427 z_zcr = w * robust_z(zcr, audible)
428 z_low = robust_z(low_db, audible_low)
429 z_low_onset = w_low * robust_z(low_onset, audible_low)
430
431 onset_gated = w * onset
432
433 # Center advantage is meaningless when both sides are near silence.
434 center_advantage = center_db - noncenter_db

```

```

436 center_advantage = np.where(
437     np.maximum(center_db, noncenter_db) > args.gate_floor,
438     center_advantage, np.nan)
439
440
441 # =====
442 # IMPACT / HARSH / DRUM / BARRAGE (gated)
443 # =====
444
445 impact_score = (0.50 * z_onset + 0.20 * w * z_loud + 0.15 * z_centroid
446     + 0.10 * z_flatness + 0.05 * z_zcr)
447
448 harsh_score = (0.30 * z_centroid + 0.30 * z_flatness + 0.20 * z_zcr
449     + 0.20 * w * z_loud)
450
451
452 def moving_mean(x, seconds):
453     k = max(1, int(seconds * frame_rate))
454     return np.convolve(x, np.ones(k) / k, mode="same")
455
456
457 drum_density = moving_mean(np.maximum(z_low_onset, 0), 4.0)
458 drum_score = 0.60 * robust_z(drum_density, audible_low) + 0.40 * w_low * z_low
459
460
461 barrage_density = moving_mean(np.maximum(z_onset, 0), 5.0)
462 barrage_score = (0.60 * robust_z(barrage_density, audible)
463     + 0.25 * w * z_loud + 0.15 * z_centroid)
464
465
466 # =====
467 # ACOUSTIC BURDEN (gated)
468 # =====
469
470 burden = (0.25 * normalize01(loud_db, audible)
471     + 0.20 * normalize01(onset_gated, audible)
472     + 0.20 * normalize01(drum_density, audible_low)
473     + 0.15 * w * normalize01(centroid, audible)
474     + 0.10 * w * normalize01(flatness, audible)
475     + 0.10 * normalize01(barrage_density, audible))
476
477 burden_smooth = gaussian_filter1d(burden, sigma=max(1, 0.5 * frame_rate))
478
479 # =====
480 # SWELL events
481 #
482 #  $S(t, D) = L(t + D) - L(t)$ 
483 #
484 # over 0.25 s energy-averaged bins, median-filtered across 1 s so a
485 # single transient cannot carry the rise. A candidate must:
486 #
487 # rise >= --swell-min-rise dB
488 # up-steps >= 60% of steps in the window
489 # top two steps together <= 50% of the total rise
490 # end level above the gate floor
491 #
492 # The top-two rule stops a hard cut from silence, smeared across two
493 # bins by the analysis window, from registering as a swell.
494 # =====
495
496 BIN = 0.25
497 bin_index = np.floor(times / BIN).astype(int)
498 n_bins = bin_index.max() + 1
499
500 power = 10.0 ** (loud_db / 10.0)
501 bin_power = np.bincount(bin_index, weights=power, minlength=n_bins)

```

```

502 bin_count = np.bincount(bin_index, minlength=n_bins)
503 bin_db = 10.0 * np.log10(np.maximum(bin_power / np.maximum(bin_count, 1), 1e-12))
504 bin_db = median_filter(bin_db, size=5, mode="nearest")
505 bin_times = np.arange(n_bins) * BIN
506
507 steps = np.diff(bin_db, prepend=bin_db[0])
508 windows = [float(x) for x in args.swell_windows.split(",") if x.strip()]
509
510 best_rise = np.zeros(n_bins)
511 best_window = np.zeros(n_bins)
512
513 for D in windows:
514     k = int(round(D / BIN))
515     if k < 2 or k >= n_bins:
516         continue
517
518     rise = np.full(n_bins, -np.inf)
519     rise[:-k] = bin_db[k:] - bin_db[:-k]
520
521     step_windows = np.lib.stride_tricks.sliding_window_view(steps[1:], k)
522     frac_up = np.zeros(n_bins)
523     top_two = np.full(n_bins, np.inf)
524     m = len(step_windows)
525     frac_up[:m] = np.mean(step_windows > 0, axis=1)
526     top_two[:m] = np.sum(np.sort(step_windows, axis=1)[: , -2:], axis=1)
527
528     end_level = np.full(n_bins, -np.inf)
529     end_level[:-k] = bin_db[k:]
530
531     ok = ((rise >= args.swell_min_rise)
532           & (frac_up >= 0.60)
533           & (top_two <= 0.50 * rise)
534           & (end_level > args.gate_floor))
535
536     better = ok & (rise > best_rise)
537     best_rise[better] = rise[better]
538     best_window[better] = D
539
540 swell_peaks, _ = find_peaks(best_rise, height=args.swell_min_rise,
541                             distance=max(1, int(4.0 / BIN)))
542
543 swell_events = []
544 taken_until = -np.inf
545 for p in sorted(swell_peaks, key=lambda i: -best_rise[i]):
546     t0 = bin_times[p]
547     t1 = t0 + best_window[p]
548     if any(not (t1 <= e["start_seconds"] or t0 >= e["end_seconds"])
549            for e in swell_events):
550         continue
551     swell_events.append({"start_seconds": t0, "end_seconds": t1,
552                         "rise_db": best_rise[p], "window": best_window[p]})
553 swell_events.sort(key=lambda e: e["start_seconds"])
554
555 # Frame-level trace for frames.csv: best rise beginning in this bin.
556 swell_trace = best_rise[np.minimum(bin_index, n_bins - 1)]
557
558
559 # =====
560 # Point peaks and regions
561 # =====
562
563 def get_peaks(score, percentile, min_gap_s, mask):
564     threshold = np.nanpercentile(score[mask] if mask.any() else score,
565                                  percentile)
566     peaks, _ = find_peaks(np.nan_to_num(score, nan=-np.inf),
567                           height=threshold,

```

```

568         distance=max(1, int(min_gap_s * frame_rate)))
569     return peaks[mask[peaks]]
570
571
572 impact_peaks = get_peaks(impact_score, 98.5, 0.75, audible)
573 drum_peaks = get_peaks(drum_score, 97.5, 4.0, audible_low)
574 harsh_peaks = get_peaks(harsh_score, 98.0, 3.0, audible)
575
576 barrage_regions = contiguous_regions(
577     barrage_score > np.percentile(barrage_score[audible], 85),
578     times, minimum_duration=3.0)
579 high_load_regions = contiguous_regions(
580     burden_smooth > np.percentile(burden_smooth, 80),
581     times, minimum_duration=4.0)
582 recovery_regions = contiguous_regions(
583     burden_smooth < np.percentile(burden_smooth, 25),
584     times, minimum_duration=5.0)
585
586
587 # =====
588 # Event table
589 # =====
590
591 events = []
592
593
594 def nanmean(x):
595     x = x[np.isfinite(x)]
596     return float(np.mean(x)) if len(x) else float("nan")
597
598
599 def add_event(label, t0, t1, score, section):
600     events.append({
601         "type": label,
602         "start_seconds": float(t0),
603         "end_seconds": float(t1),
604         "start": timestamp(t0),
605         "end": timestamp(t1),
606         "duration": float(t1 - t0),
607         "score": float(score),
608         "loudness_db": nanmean(loud_db[section]),
609         "low_db": nanmean(low_db[section]),
610         "center_advantage_db": nanmean(center_advantage[section]),
611     })
612
613
614 for label, score, peaks in [("IMPACT", impact_score, impact_peaks),
615                             ("DRUM_RUN", drum_score, drum_peaks),
616                             ("HARSH", harsh_score, harsh_peaks)]:
617     for p in peaks:
618         add_event(label, times[p], times[p], score[p], slice(p, p + 1))
619
620 for e in swell_events:
621     i0 = np.searchsorted(times, e["start_seconds"])
622     i1 = np.searchsorted(times, e["end_seconds"])
623     add_event("SWELL", e["start_seconds"], e["end_seconds"],
624             e["rise_db"], slice(i0, max(i1, i0 + 1)))
625
626 for label, score, regions in [("BARRAGE", barrage_score, barrage_regions),
627                               ("HIGH_LOAD", burden_smooth, high_load_regions),
628                               ("RECOVERY", -burden_smooth, recovery_regions)]:
629     for start, end, t0, t1 in regions:
630         add_event(label, t0, t1, np.mean(score[start:end]),
631                 slice(start, end))
632
633 events_df = pd.DataFrame(events).sort_values(["start_seconds", "type"])

```

```

634 events_df.to_csv(out / "events.csv", index=False)
635
636
637 # =====
638 # Frame and per-second tables
639 # =====
640
641 df["audibility"] = w
642 df["impact_score"] = impact_score
643 df["drum_score"] = drum_score
644 df["swell_rise_db"] = swell_trace
645 df["harsh_score"] = harsh_score
646 df["barrage_score"] = barrage_score
647 df["burden"] = burden_smooth
648 df["center_advantage_db"] = center_advantage
649 df.to_csv(out / "frames.csv", index=False)
650
651 df["second"] = np.floor(df["time"]).astype(int)
652 timeline = df.groupby("second").mean(numeric_only=True).reset_index()
653 timeline["timestamp"] = timeline["second"].map(timestamp)
654 timeline.to_csv(out / "timeline.csv", index=False)
655
656
657 # =====
658 # Summary
659 # =====
660
661 def total(regions):
662     return sum(t1 - t0 for _, _, t0, t1 in regions)
663
664
665 recovery_gaps = []
666 previous_end = 0.0
667 for _, _, t0, t1 in recovery_regions:
668     if t0 > previous_end:
669         recovery_gaps.append(t0 - previous_end)
670         previous_end = t1
671 if previous_end < duration:
672     recovery_gaps.append(duration - previous_end)
673
674 summary_lines = [
675     f"Source: {source}",
676     f"Duration: {timestamp(duration)}",
677     f"Source channels: {channels}",
678     f"Gate: {args.gate_floor:.0f} -> {args.gate_full:.0f} dBFS ",
679     f"(low: {args.low_gate_floor:.0f} -> {args.low_gate_full:.0f})",
680     f"Frames below gate floor: {100 * silent_fraction:.1f}%",
681     "",
682     f"Impact candidates: {len(impact_peaks)}",
683     f"Drum-run candidates: {len(drum_peaks)}",
684     f"Swell events: {len(swell_events)}",
685     f"Harsh candidates: {len(harsh_peaks)}",
686     "",
687     f"Barrage regions: {len(barrage_regions)}",
688     f"High-load regions: {len(high_load_regions)}",
689     f"Recovery regions: {len(recovery_regions)}",
690     "",
691     f"Barrage time: {total(barrage_regions):.1f} s ",
692     f"({100 * total(barrage_regions) / duration:.1f}%)",
693     f"High-load time: {total(high_load_regions):.1f} s ",
694     f"({100 * total(high_load_regions) / duration:.1f}%)",
695     f"Recovery time: {total(recovery_regions):.1f} s ",
696     f"({100 * total(recovery_regions) / duration:.1f}%)",
697 ]
698 if recovery_gaps:
699     summary_lines += [

```

```

700     "",
701     f"Median interval without qualifying recovery: "
702     f"{np.median(recovery_gaps):.1f} s",
703     f"Longest interval without qualifying recovery: "
704     f"{np.max(recovery_gaps):.1f} s",
705 ]
706
707 summary = "\n".join(summary_lines) + "\n"
708 (out / "summary.txt").write_text(summary)
709 print(summary)
710
711
712 # =====
713 # Plots
714 # =====
715
716 minutes = times / 60.0
717
718
719 def save_plot(values, name, ylabel, filename, spans=None, shade_silence=True):
720     plt.figure(figsize=(18, 5))
721     if shade_silence:
722         for _, _, t0, t1 in contiguous_regions(w == 0, times, 0.5):
723             plt.axvspan(t0 / 60, t1 / 60, color="0.85", zorder=0)
724     if spans:
725         for t0, t1 in spans:
726             plt.axvspan(t0 / 60, t1 / 60, color="tab:orange", alpha=0.35)
727     plt.plot(minutes, values, linewidth=0.8)
728     plt.xlabel("Movie time (minutes)")
729     plt.ylabel(ylabel)
730     plt.title(f"{title} {name}")
731     plt.tight_layout()
732     plt.savefig(out / filename, dpi=160)
733     plt.close()
734
735
736 swell_spans = [(e["start_seconds"], e["end_seconds"]) for e in swell_events]
737
738 save_plot(loud_db, "Loudness (orange: swell events, grey: below gate)",
739           "dBFS", "01-loudness.png", spans=swell_spans)
740 save_plot(onset_gated, "Acoustic Transients (level-gated)",
741           "Onset strength x audibility", "02-transients.png")
742 save_plot(drum_score, "Low-Frequency Onset Activity",
743           "Drum-run score", "03-drum-runs.png")
744
745 plt.figure(figsize=(18, 5))
746 if swell_events:
747     starts = np.array([e["start_seconds"] for e in swell_events]) / 60
748     rises = np.array([e["rise_db"] for e in swell_events])
749     wins = np.array([e["window"] for e in swell_events]) / 60
750     plt.bar(starts, rises, width=wins, align="edge", alpha=0.7)
751 plt.xlim(0, duration / 60)
752 plt.xlabel("Movie time (minutes)")
753 plt.ylabel("Sustained rise (dB)")
754 plt.title(f"{title} Swell Events (bar width = rise window)")
755 plt.tight_layout()
756 plt.savefig(out / "04-swells.png", dpi=160)
757 plt.close()
758
759 save_plot(barrage_score, "Transient Barrage (level-gated)",
760           "Barrage score", "05-barrage.png")
761 save_plot(harsh_score, "Spectral Harshness (level-gated)",
762           "Harshness score", "06-harshness.png")
763 save_plot(burden_smooth, "Acoustic Burden (level-gated)",
764           "Burden", "07-burden.png")
765 save_plot(center_advantage, "Center-Channel Advantage (NaN below gate)",

```

```

766         "Center minus non-center (dB)", "08-center-advantage.png")
767 save_plot(burden_smooth, "Recovery Windows", "Acoustic burden",
768         "09-recovery.png",
769         spans=[(t0, t1) for _, _, t0, t1 in recovery_regions])
770
771 plt.figure(figsize=(18, 6))
772 plt.plot(minutes, normalize01(drum_score) + 3, linewidth=0.8, color="tab:blue")
773 for t0, t1 in swell_spans:
774     plt.axvspan(t0 / 60, t1 / 60, ymin=0.5, ymax=0.75,
775               color="tab:orange", alpha=0.8)
776 plt.plot(minutes, normalize01(barrage_score) + 1, linewidth=0.8, color="tab:green")
777 plt.plot(minutes, normalize01(burden_smooth), linewidth=0.8, color="tab:red")
778 plt.yticks([0.5, 1.5, 2.5, 3.5], ["Burden", "Barrage", "Swell", "Drums"])
779 plt.ylim(0, 4)
780 plt.xlabel("Movie time (minutes)")
781 plt.title(f"{title} Sound Grammar")
782 plt.tight_layout()
783 plt.savefig(out / "10-sound-grammar.png", dpi=160)
784 plt.close()
785
786
787 # =====
788 # Top candidates
789 # =====
790
791 report = []
792 for event_type in ["DRUM_RUN", "SWELL", "IMPACT", "HARSH", "BARRAGE"]:
793     subset = (events_df[events_df["type"] == event_type]
794             .sort_values("score", ascending=False).head(20))
795     report.append(f"\n{event_type}\n" + "=" * len(event_type))
796     for _, row in subset.iterrows():
797         report.append(
798             f"{row['start']} score={row['score']:.2f} "
799             f"dur={row['duration']:.1f}s level={row['loudness_db']:.1f} dB "
800             f"low={row['low_db']:.1f} dB "
801             f"center={row['center_advantage_db']:.1f} dB")
802 (out / "top-candidates.txt").write_text("\n".join(report) + "\n")
803
804
805 # =====
806 # Audition clips
807 # =====
808
809 def clip_window(row):
810     t0, t1 = row["start_seconds"], row["end_seconds"]
811     if row["type"] == "SWELL":
812         a, b = t0 - 4.0, t1 + 6.0
813     elif row["type"] in ("BARRAGE", "HIGH_LOAD"):
814         a, b = t0 - 5.0, min(t1, t0 + 20.0) + 5.0
815     else:
816         a, b = t0 - args.pre, t0 + args.post
817     a = max(0.0, a)
818     b = min(duration, b, a + 30.0)
819     return a, b
820
821
822 if args.audition > 0:
823     print("Cutting audition clips...")
824     aud = out / "audition"
825     aud.mkdir(exist_ok=True)
826     rows = []
827
828     for event_type in ["DRUM_RUN", "SWELL", "IMPACT", "HARSH", "BARRAGE"]:
829         subset = (events_df[events_df["type"] == event_type]
830                 .sort_values("score", ascending=False)
831                 .head(args.audition))

```

```

832     folder = aud / event_type.lower().replace("_run", "")
833     folder.mkdir(exist_ok=True)
834     playlist = []
835
836     for rank, (_, row) in enumerate(subset.iterrows(), start=1):
837         a, b = clip_window(row)
838         name = (f"{rank:02d}_{file_stamp(row['start_seconds'])}"
839               f"_score-{row['score']:.2f}.mp3")
840         target = folder / name
841         if not target.exists():
842             run(["ffmpeg", "-y", "-hide_banner", "-loglevel", "error",
843                 "-ss", f"{a:.3f}", "-i", str(source),
844                 "-t", f"{b - a:.3f}", "-map", args.audio_stream,
845                 "-vn", "-ac", "2", "-c:a", "libmp3lame", "-q:a", "4",
846                 str(target)], quiet=True)
847         playlist.append(name)
848         rows.append({
849             "clip": f"{folder.name}/{name}",
850             "category": event_type,
851             "rank": rank,
852             "event_time": timestamp(row["start_seconds"]),
853             "event_seconds": round(float(row["start_seconds"]), 2),
854             "event_offset_in_clip": round(row["start_seconds"] - a, 2),
855             "score": round(float(row["score"]), 3),
856             "level_db": round(float(row["loudness_db"]), 1),
857             "label": "",
858             "role": "",
859             "intensity": "",
860             "would_reduce": "",
861             "notes": "",
862         })
863     (folder / "playlist.m3u").write_text("\n".join(playlist) + "\n")
864     print(f" {event_type:9s} {len(playlist)} clips")
865
866     ann = aud / "annotations.csv"
867     target = ann if not ann.exists() else aud / "annotations.new.csv"
868     pd.DataFrame(rows).to_csv(target, index=False)
869     (aud / "LABELS.txt").write_text(
870         "Fill the `label` column with one of (or several joined by '+'):\n\n"
871         + "\n".join(LABELS) + "\n")
872     if target != ann:
873         print(f" {ann} exists; new template written to {target.name}")
874
875
876     # =====
877     # Annotation summary
878     # =====
879
880     if args.annotations:
881         ann = pd.read_csv(args.annotations).fillna("")
882         ann = ann[ann["label"].str.strip() != ""]
883         if len(ann) == 0:
884             print("No labelled rows in", args.annotations)
885         else:
886             ann["labels"] = ann["label"].str.upper().str.split("+")
887             exploded = ann.explode("labels").reset_index(drop=True)
888             exploded["labels"] = exploded["labels"].str.strip()
889
890             lines = ["DETECTOR PRECISION (share not FALSE_POSITIVE)", ""]
891             for cat, g in ann.groupby("category"):
892                 fp = g["labels"].apply(lambda L: "FALSE_POSITIVE" in L).sum()
893                 lines.append(f"{cat:9s} {len(g) - fp:3d}/{len(g):3d} "
894                             f" f"= {(len(g) - fp) / len(g):.0%}")
895             lines += [",", "WHAT EACH DETECTOR ACTUALLY FOUND", ""]
896             table = pd.crosstab(exploded["category"], exploded["labels"])
897             lines.append(table.to_string())

```

```

898     text = "\n".join(lines) + "\n"
899     (out / "annotation-summary.txt").write_text(text)
900     print(text)
901
902     kinds = sorted(set(exploded["labels"]) - {"FALSE_POSITIVE"})
903     plt.figure(figsize=(18, 0.5 * len(kinds) + 2))
904     for i, k in enumerate(kinds):
905         ts = exploded.loc[exploded["labels"] == k, "event_seconds"] / 60
906         plt.scatter(ts, np.full(len(ts), i), marker="|", s=300)
907     plt.yticks(range(len(kinds)), kinds)
908     plt.xlim(0, duration / 60)
909     plt.xlabel("Movie time (minutes)")
910     plt.title(f"{title} Labelled Events")
911     plt.tight_layout()
912     plt.savefig(out / "11-labelled-events.png", dpi=160)
913     plt.close()
914
915
916 print("\nWritten to", out)
917 print("Done.")

```

C.2 Dialogue-referenced occupation analysis

```

1  #!/usr/bin/env python3
2  """
3  occupation.py (v2)
4  =====
5
6  Dialogue-referenced occupation analysis for Acoustic Occupation.
7
8  Inputs
9      --frames frames.csv from analyze-soundtrack-v3.1
10     --events events.csv from analyze-soundtrack-v3.1
11     --film the film itself (optional; enables BS.1770 loudness)
12     --subtitles an.srt aligned with the film (optional; enables a
13         subtitle-defined dialogue reference and validation of
14         the centre-channel proxy)
15
16  Outputs (in --out)
17     results.tex LaTeX macros consumed by the manuscript
18     occupation.json the same numbers, machine readable
19     seconds.csv per-second table used for everything below
20     loudness.csv cached BS.1770 momentary loudness (if --film)
21     fig-occupation.png level, dialogue reference, blocks, subtitle rug
22     fig-exceedance.png distribution of level relative to dialogue
23     fig-blocks.png level-defined vs transient-defined blocks
24
25  Changes from v1
26     * Level can be BS.1770 loudness (ffmpeg ebur128, momentary values
27       energy-averaged per second) instead of unweighted RMS of a mono
28       downmix. Integrated loudness and loudness range are reported.
29     * Subtitle cue timings define a second dialogue reference. SDH cues
30       that describe sound rather than speech ([GUNFIRE], (sighs),
31       music notes (U+266A, U+266B)) are removed. The centre-channel proxy is validated
32       against subtitle coverage (precision and recall).
33     * Subtitle offset can be estimated automatically by lagging cue
34       coverage against centre-channel dominance.
35     * The overdetermination index is computed both raw and with each
36       channel residualised on level, so co-elevation that level alone
37       explains is not counted.
38     * Reports how much subtitled dialogue falls inside occupation
39       blocks, i.e. dialogue exposed to being turned down or off.

```

```

40 """
41
42 import argparse
43 import html
44 import json
45 import re
46 import subprocess
47 from pathlib import Path
48
49 import matplotlib
50 matplotlib.use("Agg")
51 import matplotlib.pyplot as plt
52 import numpy as np
53 import pandas as pd
54
55
56 p = argparse.ArgumentParser()
57 p.add_argument("--frames", required=True)
58 p.add_argument("--events", required=True)
59 p.add_argument("--out", default="results")
60 p.add_argument("--film", default=None,
61               help="film file; enables BS.1770 loudness via ffmpeg ebur128")
62 p.add_argument("--audio-stream", default="0:a:0")
63 p.add_argument("--level", choices=["auto", "lufs", "rms"], default="auto",
64               help="auto = lufs when --film is given, else rms")
65 p.add_argument("--subtitles", default=None, help=".srt file")
66 p.add_argument("--sub-offset", default="auto",
67               help="seconds to ADD to subtitle times, or 'auto', or '0'")
68 p.add_argument("--reference", choices=["auto", "subtitles", "proxy"],
69               default="auto",
70               help="auto = subtitles when available, else proxy")
71 p.add_argument("--delta", type=float, default=10.0)
72 p.add_argument("--merge-gap", type=float, default=5.0)
73 p.add_argument("--min-block", type=float, default=60.0)
74 p.add_argument("--center-adv", type=float, default=6.0)
75 p.add_argument("--dialogue-barrage-pct", type=float, default=75.0)
76 p.add_argument("--gate-floor", type=float, default=-65.0,
77               help="audibility floor for RMS level (dBFS)")
78 p.add_argument("--lufs-floor", type=float, default=-60.0,
79               help="audibility floor for BS.1770 level (LUFS)")
80 p.add_argument("--recovery-margin", type=float, default=0.0)
81 p.add_argument("--recovery-width", type=float, default=5.0)
82 p.add_argument("--channel-z", type=float, default=1.5)
83 args = p.parse_args()
84
85 out = Path(args.out)
86 out.mkdir(parents=True, exist_ok=True)
87
88
89 # =====
90 # Helpers
91 # =====
92
93 def ts(sec):
94     sec = max(0, int(round(sec)))
95     return f"{sec // 3600:d}:{(sec % 3600) // 60:02d}:{sec % 60:02d}"
96
97
98 def energy_mean_db(x):
99     x = np.asarray(x, dtype=float)
100     x = x[np.isfinite(x)]
101     if len(x) == 0:
102         return np.nan
103     return 10 * np.log10(np.mean(10 ** (x / 10)))
104
105

```

```

106 def runs(mask):
107     mask = np.asarray(mask, dtype=bool)
108     if not mask.any():
109         return []
110     d = np.diff(np.concatenate([[0], mask.astype(int), [0]]))
111     return list(zip(np.where(d == 1)[0], np.where(d == -1)[0]))
112
113
114 def merge(intervals, gap):
115     merged = []
116     for a, b in sorted(intervals):
117         if merged and a - merged[-1][1] <= gap:
118             merged[-1][1] = max(merged[-1][1], b)
119         else:
120             merged.append([a, b])
121     return [(a, b) for a, b in merged]
122
123
124 def pct(x):
125     return f"{100 * x:.1f}\\%" if np.isfinite(x) else "--"
126
127
128 def robust_scale(x):
129     x = np.asarray(x, float)
130     x = x[np.isfinite(x)]
131     if len(x) == 0:
132         return 1.0
133     s = 1.4826 * np.median(np.abs(x - np.median(x)))
134     if s < 1e-12:
135         s = np.std(x)
136     return s if s > 1e-12 else 1.0
137
138
139 # =====
140 # BS.1770 loudness via ffmpeg ebur128
141 # =====
142
143 EBU_LINE = re.compile(r"t:\s*([\d.]+).*?M:\s*(-?[\d.]+|-?inf|nan)\s+"
144                       r"S:\s*(-?[\d.]+|-?inf|nan)")
145
146
147 def bs1770(film):
148     cache = out / "loudness.csv"
149     summ = out / "loudness-summary.json"
150     if cache.exists() and summ.exists():
151         print("Reusing", cache)
152         return pd.read_csv(cache), json.loads(summ.read_text())
153
154     print("Measuring BS.1770 loudness (one decode pass)...")
155     cmd = ["ffmpeg", "-hide_banner", "-nostats", "-i", film,
156           "-map", args.audio_stream, "-vn",
157           "-af", "ebur128=framelog=info", "-f", "null", "-"]
158     proc = subprocess.Popen(cmd, stderr=subprocess.PIPE, text=True,
159                             errors="replace")
160
161     t, m, s = [], [], []
162     tail = []
163     for line in proc.stderr:
164         mm = EBU_LINE.search(line)
165         if mm:
166             t.append(float(mm.group(1)))
167             m.append(-120.0 if "inf" in mm.group(2) else float(mm.group(2)))
168             s.append(float(mm.group(3)) if "inf" not in mm.group(3) else -120.0)
169         else:
170             tail.append(line)
171             tail = tail[-40:]
172     proc.wait()

```

```

172     if proc.returncode != 0 or not t:
173         raise SystemExit("ebur128 measurement failed:\n" + "".join(tail))
174
175     text = "".join(tail)
176
177     def grab(label):
178         r = re.search(label + r":\s*(-?[\d.]+)", text)
179         return float(r.group(1)) if r else float("nan")
180
181     summary = {"integrated_lufs": grab("I"), "lra_lu": grab("LRA"),
182               "lra_low": grab("LRA low"), "lra_high": grab("LRA high")}
183     df = pd.DataFrame({"time": t, "momentary": m, "short_term": s})
184     df["momentary"] = df["momentary"].clip(lower=-120)
185     df.to_csv(cache, index=False)
186     summ.write_text(json.dumps(summary, indent=2))
187     return df, summary
188
189
190 # =====
191 # Subtitles
192 # =====
193
194 SRT_TIME = re.compile(r"(\d+):(\d+):(\d+)[,.] (\d+)\s*-->\s*(\d+):(\d+):(\d+)[,.] (\d+)")
195 TAG = re.compile(r"<[>]+>|\{[~]*\}+")
196 NONSPEECH = re.compile(r"^\s*(\[ [^]]*\] | \([^\)]*\) | [\u266a\u266b#\s-]+)+\s*$")
197
198
199 def parse_srt(path):
200     raw = Path(path).read_bytes()
201     for enc in ("utf-8-sig", "cp1252", "latin-1"):
202         try:
203             text = raw.decode(enc)
204             break
205         except UnicodeDecodeError:
206             continue
207     cues = []
208     for block in re.split(r"\r?\n\s*\r?\n", text.strip()):
209         lines = block.strip().splitlines()
210         for i, line in enumerate(lines):
211             mt = SRT_TIME.search(line)
212             if mt:
213                 g = [int(x) for x in mt.groups()]
214                 a = g[0] * 3600 + g[1] * 60 + g[2] + g[3] / 1000
215                 b = g[4] * 3600 + g[5] * 60 + g[6] + g[7] / 1000
216                 body = " ".join(lines[i + 1:])
217                 body = html.unescape(TAG.sub("", body)).strip()
218                 cues.append((a, b, body))
219             break
220     return cues
221
222
223 def is_speech(body):
224     if not body:
225         return False
226     if NONSPEECH.match(body):
227         return False
228     # Strip bracketed sound descriptions and speaker labels, then see
229     # whether any words remain.
230     rest = re.sub(r"([^\]]*\] | \([^\)]*\) | [\u266a~\u266b#\u266a]^[A-Z .'-]+\s*", "", body)
231     return bool(re.search(r"[A-Za-z]{2,}", rest))
232
233
234 def coverage(cues, n_sec, offset):
235     cov = np.zeros(n_sec)
236     for a, b, _ in cues:
237         a, b = a + offset, b + offset

```

```

238     if b <= 0 or a >= n_sec:
239         continue
240     a, b = max(0.0, a), min(float(n_sec), b)
241     i0, i1 = int(np.floor(a)), int(np.ceil(b))
242     for i in range(i0, min(i1, n_sec)):
243         cov[i] += max(0.0, min(b, i + 1) - max(a, i))
244     return np.clip(cov, 0, 1)
245
246
247 # =====
248 # Per-second table
249 # =====
250
251 f = pd.read_csv(args.frames)
252 e = pd.read_csv(args.events)
253 f["second"] = np.floor(f["time"]).astype(int)
254 g = f.groupby("second")
255
256 sec = pd.DataFrame({
257     "rms_db": g["loudness_db"].apply(energy_mean_db),
258     "center_db": g["center_db"].apply(energy_mean_db),
259     "noncenter_db": g["noncenter_db"].apply(energy_mean_db),
260     "barrage": g["barrage_score"].mean(),
261     "drum": g["drum_score"].mean(),
262     "harsh": g["harsh_score"].max(),
263     "burden": g["burden"].mean(),
264 }).reset_index()
265 sec = sec.set_index("second").reindex(
266     range(int(sec["second"].max()) + 1)).rename_axis("second").reset_index()
267 sec["center_adv"] = sec["center_db"] - sec["noncenter_db"]
268 n_sec = len(sec)
269 duration = float(f["time"].max())
270
271 level_mode = args.level
272 if level_mode == "auto":
273     level_mode = "lufs" if args.film else "rms"
274
275 loud_summary = {}
276 if level_mode == "lufs":
277     if not args.film:
278         raise SystemExit("--level lufs needs --film")
279     ldf, loud_summary = bs1770(args.film)
280     ldf["second"] = np.floor(ldf["time"] - 1e-6).astype(int)
281     lufs_sec = ldf.groupby("second")["momentary"].apply(energy_mean_db)
282     sec["lufs"] = sec["second"].map(lufs_sec)
283     sec["level_db"] = sec["lufs"]
284     floor = args.lufs_floor
285     unit = "LUFS"
286 else:
287     sec["level_db"] = sec["rms_db"]
288     floor = args.gate_floor
289     unit = "dBFS"
290
291 sec["audible"] = sec["level_db"] > floor
292
293
294 # =====
295 # Dialogue references
296 # =====
297
298 candidate = sec["audible"] & (sec["center_adv"] >= args.center_adv)
299 cap = (np.percentile(sec.loc[candidate, "barrage"], args.dialogue_barrage_pct)
300        if candidate.any() else -np.inf)
301 proxy = candidate & (sec["barrage"] <= cap)
302 sec["dialogue_proxy"] = proxy
303 D_proxy = float(sec.loc[proxy, "level_db"].median()) if proxy.sum() >= 30 else float("nan")

```

```

304
305 sub = {"available": False}
306 if args.subtitles and Path(args.subtitles).exists():
307     cues_all = parse_srt(args.subtitles)
308     cues = [c for c in cues_all if is_speech(c[2])]
309
310     centre_dom = (sec["audible"] & (sec["center_adv"] >= args.center_adv)
311                 ).to_numpy().astype(float)
312
313     def corr_at(off):
314         c = coverage(cues, n_sec, off)
315         if c.std() == 0 or centre_dom.std() == 0:
316             return 0.0
317         return float(np.corrcoef(c, centre_dom)[0, 1])
318
319     r0 = corr_at(0.0)
320     if args.sub_offset == "auto":
321         lags = np.arange(-30.0, 30.5, 0.5)
322         rs = [corr_at(x) for x in lags]
323         best = float(lags[int(np.argmax(rs))])
324         rbest = float(max(rs))
325         offset = best if (rbest - r0) > 0.1 * max(abs(r0), 1e-3) and abs(best) >= 0.5 else 0.0
326     else:
327         offset = float(args.sub_offset)
328         rbest = corr_at(offset)
329     cov = coverage(cues, n_sec, offset)
330     sec["sub_coverage"] = cov
331     sub_sec = sec["audible"] & (cov >= 0.5)
332     sec["dialogue_sub"] = sub_sec
333
334     D_sub = float(sec.loc[sub_sec, "level_db"].median()) if sub_sec.sum() >= 30 else float("nan")
335     capped = sub_sec & (sec["barrage"] <= np.percentile(
336         sec.loc[sub_sec, "barrage"], 75)) if sub_sec.any() else sub_sec
337     D_sub_capped = float(sec.loc[capped, "level_db"].median()) if capped.sum() >= 30 else float("nan")
338
339     tp = int((proxy & sub_sec).sum())
340     sub.update({
341         "available": True,
342         "cues_total": len(cues_all),
343         "cues_speech": len(cues),
344         "cues_nonspeech": len(cues_all) - len(cues),
345         "offset_s": offset,
346         "corr_at_zero": r0,
347         "corr_at_offset": rbest if args.sub_offset == "auto" else corr_at(offset),
348         "seconds": int(sub_sec.sum()),
349         "D": D_sub,
350         "D_capped": D_sub_capped,
351         "proxy_precision": tp / max(1, int(proxy.sum())),
352         "proxy_recall": tp / max(1, int(sub_sec.sum())),
353     })
354 else:
355     sec["sub_coverage"] = 0.0
356     sec["dialogue_sub"] = False
357
358 ref = args.reference
359 if ref == "auto":
360     ref = "subtitles" if sub["available"] and np.isfinite(sub.get("D", np.nan)) else "proxy"
361 if ref == "subtitles":
362     D = sub["D"]
363     D_source = "subtitle-defined dialogue seconds"
364     dialogue = sec["dialogue_sub"]
365 else:
366     D = D_proxy
367     D_source = "centre-channel proxy"
368     dialogue = sec["dialogue_proxy"]
369 fallback = not np.isfinite(D)

```

```

370 if fallback:
371     D = float(sec.loc[sec["audible"], "level_db"].median())
372     D_source = "all audible seconds (too few dialogue seconds)"
373
374 D_center = float(sec.loc[dialogue, "center_db"].median()) if dialogue.any() else float("nan")
375
376 rel = sec["level_db"] - D
377 sec["rel_db"] = rel
378 aud_rel = rel[sec["audible"]].dropna()
379 exceed = {k: float(np.mean(aud_rel >= k)) for k in (5, 10, 15, 20)}
380 p95 = float(np.percentile(aud_rel, 95))
381 p99 = float(np.percentile(aud_rel, 99))
382
383
384 # =====
385 # Occupation blocks
386 # =====
387
388 occ = (rel >= args.delta).fillna(False).to_numpy()
389 sec["occupied"] = occ
390 occ_runs = [(int(sec["second"][a]), int(sec["second"][b - 1]) + 1) for a, b in runs(occ)]
391 occ_merged = merge(occ_runs, args.merge_gap)
392 occ_blocks = [(a, b) for a, b in occ_merged if b - a >= args.min_block]
393
394 in_block = np.zeros(n_sec, dtype=bool)
395 for a, b in occ_blocks:
396     in_block[a:b] = True
397 sec["in_block"] = in_block
398
399 block_rows = []
400 for a, b in occ_blocks:
401     r = rel.iloc[a:b].dropna()
402     block_rows.append({
403         "start": float(a), "end": float(b), "duration": float(b - a),
404         "occupied_share": float(np.mean(r >= args.delta)),
405         "median_rel": float(np.median(r)),
406         "p95_rel": float(np.percentile(r, 95)),
407         "implied_throttle_db": float(max(0.0, np.percentile(r, 95) - args.delta)),
408         "sub_seconds": int(sec["dialogue_sub"].iloc[a:b].sum()),
409     })
410
411 # Dialogue exposed to throttling: subtitled speech inside blocks.
412 sub_in_blocks = int((sec["dialogue_sub"] & in_block).sum())
413 sub_share_in_blocks = (sub_in_blocks / max(1, sub.get("seconds", 0))
414                        if sub["available"] else float("nan"))
415
416
417 # =====
418 # Transient-defined blocks
419 # =====
420
421 bar = e[e["type"] == "BARRAGE"]
422 bar_merged = merge(list(zip(bar["start_seconds"], bar["end_seconds"])), args.merge_gap)
423 bar_blocks = [(a, b) for a, b in bar_merged if b - a >= args.min_block]
424
425
426 # =====
427 # Recovery
428 # =====
429
430 rec = e[e["type"] == "RECOVERY"].sort_values("start_seconds")
431 gaps, prev = [], 0.0
432 for _, r in rec.iterrows():
433     if r["start_seconds"] > prev:
434         gaps.append((prev, r["start_seconds"]))
435     prev = r["end_seconds"]

```

```

436 if prev < duration:
437     gaps.append((prev, duration))
438 gaps.sort(key=lambda x: x[1] - x[0], reverse=True)
439
440 calm = ((rel <= args.recovery_margin) | ~sec["audible"]).fillna(True).to_numpy()
441 abs_windows = [(int(sec["second"][a]), int(sec["second"][b - 1]) + 1)
442                 for a, b in runs(calm) if b - a >= args.recovery_width]
443 abs_gaps, prev = [], 0.0
444 for a, b in abs_windows:
445     if a > prev:
446         abs_gaps.append((prev, a))
447     prev = b
448 if prev < duration:
449     abs_gaps.append((prev, duration))
450 abs_gaps.sort(key=lambda x: x[1] - x[0], reverse=True)
451 abs_share = sum(b - a for a, b in abs_windows) / max(1.0, duration)
452
453 in_window = np.zeros(n_sec, dtype=bool)
454 for a, b in abs_windows:
455     in_window[a:b] = True
456 excess = np.nan_to_num(np.maximum(0.0, rel.to_numpy() - args.delta))
457 debt, k = np.zeros(n_sec), 0.0
458 for i in range(n_sec):
459     k = 0.0 if in_window[i] else k + excess[i]
460     debt[i] = k
461 sec["recovery_debt"] = debt
462
463
464 # =====
465 # Swells
466 # =====
467
468 sw = e[e["type"] == "SWELL"].copy()
469 lvl = sec.set_index("second")["level_db"]
470 sw["start_level"] = sw["start_seconds"].map(lambda t: float(lvl.get(int(t), np.nan)))
471 sw["end_level"] = sw["end_seconds"].map(lambda t: float(lvl.get(int(t), np.nan)))
472 quiet_start = floor + 5
473 swell_n = len(sw)
474 swell_from_quiet = int((sw["start_level"] < quiet_start).sum())
475 swell_into_occ = int((sw["end_level"] >= D + args.delta).sum())
476 swell_large = int((sw["score"] >= 20).sum())
477
478 swell_active = np.zeros(n_sec, dtype=bool)
479 for _, r in sw.iterrows():
480     swell_active[int(r["start_seconds"]):int(r["end_seconds"]) + 1] = True
481
482
483 # =====
484 # Overdetermination: raw and residualised on level
485 # =====
486
487 aud = sec["audible"].fillna(False).to_numpy()
488 z = args.channel_z
489 chan = {"barrage": sec["barrage"].to_numpy(),
490         "drum": sec["drum"].to_numpy(),
491         "harsh": sec["harsh"].to_numpy()}
492
493 raw_elev = {c: np.nan_to_num(v) > z for c, v in chan.items()}
494
495 resid_elev = {}
496 L = sec["level_db"].to_numpy()
497 for c, v in chan.items():
498     ok = aud & np.isfinite(v) & np.isfinite(L)
499     coef = np.polyfit(L[ok], v[ok], 2) if ok.sum() > 10 else np.zeros(3)
500     res = v - np.polyval(coef, L)
501     # Scale by the channel's own spread, not the residual's, so that

```

```

502     # "elevated beyond what level predicts" is measured in the same
503     # units as the raw threshold.
504     # A channel counts in the residualised index only if it is elevated
505     # on the raw scale AND at least half that elevation remains after
506     # removing what level predicts. This keeps omega_resid <= omega.
507     s = robust_scale(v[ok])
508     resid_elev[c] = raw_elev[c] & (np.nan_to_num(res / s) > z / 2)
509
510 exc = np.nan_to_num(rel.to_numpy(), nan=-99) >= args.delta
511 omega = exc.astype(int) + swell_active.astype(int) + sum(x.astype(int) for x in raw_elev.values())
512 omega_r = exc.astype(int) + swell_active.astype(int) + sum(x.astype(int) for x in resid_elev.values())
513 sec["omega"] = omega
514 sec["omega_resid"] = omega_r
515
516
517 def mean_on(x, m):
518     return float(x[m].mean()) if m.any() else float("nan")
519
520
521 def share_on(x, m, k=3):
522     return float(np.mean(x[m] >= k)) if m.any() else float("nan")
523
524
525 om = {
526     "raw_in": mean_on(omega, in_block & aud), "raw_out": mean_on(omega, ~in_block & aud),
527     "raw_three": share_on(omega, aud), "raw_three_in": share_on(omega, in_block & aud),
528     "res_in": mean_on(omega_r, in_block & aud), "res_out": mean_on(omega_r, ~in_block & aud),
529     "res_three": share_on(omega_r, aud), "res_three_in": share_on(omega_r, in_block & aud),
530 }
531
532
533 # =====
534 # Figures
535 # =====
536
537 minutes = sec["second"] / 60
538
539 fig, ax = plt.subplots(figsize=(12, 4.4))
540 for a, b in occ_blocks:
541     ax.axvspan(a / 60, b / 60, color="tab:red", alpha=0.18, lw=0)
542     ax.plot(minutes, sec["level_db"], lw=0.6, color="0.25")
543     ax.axhline(D, color="tab:blue", lw=1, label=f"dialogue reference {D:.1f} {unit}")
544     ax.axhline(D + args.delta, color="tab:red", lw=1, ls="--",
545               label=f"reference + {args.delta:.0f} dB")
546     lo = np.nanmax([np.nanmin(sec["level_db"]) - 2, floor - 15])
547     hi = np.nanmax(sec["level_db"]) + 3
548     if sub["available"]:
549         subm = sec["dialogue_sub"].to_numpy()
550         ax.vlines(minutes[subm], lo, lo + 0.04 * (hi - lo), color="tab:blue",
551                 lw=0.3, label="subtitled speech")
552     ax.set_ylim(lo, hi)
553     ax.set_xlabel("Film time (minutes)")
554     ax.set_ylabel(f"Per-second level ({unit})")
555     ax.legend(loc="upper left", fontsize=7, ncol=3)
556     fig.tight_layout()
557     fig.savefig(out / "fig-occupation.png", dpi=200)
558     plt.close(fig)
559
560 plt.figure(figsize=(7, 3.5))
561 plt.hist(aud_rel, bins=np.arange(-40, 31, 1), color="0.45")
562 plt.axvline(0, color="tab:blue", lw=1)
563 plt.axvline(args.delta, color="tab:red", lw=1)
564 plt.xlabel("Per-second level minus dialogue reference (dB)")
565 plt.ylabel("Seconds")
566 plt.tight_layout()
567 plt.savefig(out / "fig-exceedance.png", dpi=200)

```

```

568 plt.close()
569
570 plt.figure(figsize=(12, 2.4))
571 for a, b in occ_merged:
572     plt.plot([a / 60, b / 60], [1, 1], lw=8, solid_capstyle="butt",
573             color="tab:red" if b - a >= args.min_block else "tab:pink")
574 for a, b in bar_merged:
575     plt.plot([a / 60, b / 60], [0, 0], lw=8, solid_capstyle="butt",
576             color="tab:green" if b - a >= args.min_block else "tab:olive")
577 plt.yticks([0, 1], ["transient-defined", "level-defined"])
578 plt.ylim(-0.7, 1.7)
579 plt.xlim(0, duration / 60)
580 plt.xlabel("Film time (minutes)")
581 plt.tight_layout()
582 plt.savefig(out / "fig-blocks.png", dpi=200)
583 plt.close()
584
585 sec.to_csv(out / "seconds.csv", index=False)
586
587
588 # =====
589 # Results
590 # =====
591
592 res = {
593     "level_measure": level_mode, "unit": unit, "loudness": loud_summary,
594     "dialogue_reference": D, "dialogue_reference_source": D_source,
595     "dialogue_reference_proxy": D_proxy, "dialogue_proxy_seconds": int(proxy.sum()),
596     "subtitles": sub, "dialogue_center_dbfs": D_center,
597     "delta_db": args.delta,
598     "recovery_margin_db": args.recovery_margin, "recovery_width_s": args.recovery_width,
599     "exceed": exceed, "p95_rel": p95, "p99_rel": p99,
600     "occupation_blocks": block_rows,
601     "barrage_blocks": [{"start": a, "end": b} for a, b in bar_blocks],
602     "recovery_gaps_top5": [{"start": a, "end": b} for a, b in gaps[:5]],
603     "abs_recovery_windows": len(abs_windows), "abs_recovery_share": abs_share,
604     "abs_gaps_top5": [{"start": a, "end": b} for a, b in abs_gaps[:5]],
605     "max_recovery_debt": float(debt.max()),
606     "subtitled_seconds_in_blocks": sub_in_blocks,
607     "subtitled_share_in_blocks": sub_share_in_blocks,
608     "swell": {"n": swell_n, "from_quiet": swell_from_quiet,
609             "into_occupied": swell_into_occ, "large": swell_large},
610     "omega": om,
611 }
612 (out / "occupation.json").write_text(json.dumps(res, indent=2, default=float))
613
614
615 def fmt(x, spec=".1f", none="--"):
616     return format(x, spec) if isinstance(x, (int, float)) and np.isfinite(x) else none
617
618
619 occ_rows = " \\\\n".join(
620     f"{ts(r['start'])} & {ts(r['end'])} & {r['duration'] / 60:.1f} & "
621     f"{r['median_rel']:.1f} & {r['p95_rel']:.1f} & {r['implied_throttle_db']:.1f}"
622     + (f" & {r['sub_seconds']}" if sub["available"] else " & --")
623     for r in block_rows) or "\\multicolumn{7}{c}{none at these settings}"
624 abs_rows = " \\\\n".join(f"{ts(a)} & {ts(b)} & {(b - a) / 60:.1f}"
625     for a, b in abs_gaps[:5]) or "\\multicolumn{3}{c}{none}"
626 gap_rows = " \\\\n".join(f"{ts(a)} & {ts(b)} & {(b - a) / 60:.1f}"
627     for a, b in gaps[:5]) or "\\multicolumn{3}{c}{none}"
628 longest = max(block_rows, key=lambda r: r["duration"]) if block_rows else None
629 thr = [r["implied_throttle_db"] for r in block_rows]
630
631 macros = {
632     "resRun": "yes",
633     "resLevelUnit": unit,

```

```

634 "resLevelMeasure": ("BS.1770 momentary loudness, energy-averaged per second"
635                     if level_mode == "lufs" else
636                     "unweighted RMS of the mono downmix, energy-averaged per second"),
637 "resLoudnessRun": "yes" if level_mode == "lufs" else "no",
638 "resIntegrated": fmt(loud_summary.get("integrated_lufs", float("nan"))),
639 "resLRA": fmt(loud_summary.get("lra_lu", float("nan"))),
640 "resLRALow": fmt(loud_summary.get("lra_low", float("nan"))),
641 "resLRAHigh": fmt(loud_summary.get("lra_high", float("nan"))),
642 "resSubRun": "yes" if sub["available"] else "no",
643 "resSubCues": str(sub.get("cues_total", "--")),
644 "resSubSpeechCues": str(sub.get("cues_speech", "--")),
645 "resSubNonspeechCues": str(sub.get("cues_nonspeech", "--")),
646 "resSubOffset": fmt(sub.get("offset_s", float("nan")), "+.1f"),
647 "resSubSeconds": str(sub.get("seconds", "--")),
648 "resSubCorrZero": fmt(sub.get("corr_at_zero", float("nan")), ".2f"),
649 "resSubCorrOffset": fmt(sub.get("corr_at_offset", float("nan")), ".2f"),
650 "resDialogueRefSub": fmt(sub.get("D", float("nan"))),
651 "resDialogueRefSubCapped": fmt(sub.get("D_capped", float("nan"))),
652 "resDialogueRefProxy": fmt(D_proxy),
653 "resProxySeconds": str(int(proxy.sum())),
654 "resProxyPrecision": pct(sub.get("proxy_precision", float("nan"))),
655 "resProxyRecall": pct(sub.get("proxy_recall", float("nan"))),
656 "resDialogueSource": D_source,
657 "resDialogueRef": fmt(D),
658 "resDialogueFallback": "yes" if fallback else "no",
659 "resDialogueCenter": fmt(D_center),
660 "resDialogueSeconds": str(int(dialogue.sum())),
661 "resDialogueShare": pct(dialogue.sum() / n_sec),
662 "resDelta": f"{args.delta:.0f}",
663 "resMergeGap": f"{args.merge_gap:.0f}",
664 "resMinBlock": f"{args.min_block:.0f}",
665 "resExceedFive": pct(exceed[5]),
666 "resExceedTen": pct(exceed[10]),
667 "resExceedFifteen": pct(exceed[15]),
668 "resExceedTwenty": pct(exceed[20]),
669 "resPNinetyFive": f"{p95:+.1f}",
670 "resPNinetyNine": f"{p99:+.1f}",
671 "resOccBlocksN": str(len(block_rows)),
672 "resOccTotalMin": f"{sum(r['duration'] for r in block_rows) / 60:.1f}",
673 "resOccLongestMin": f"{longest['duration'] / 60:.1f}" if longest else "0",
674 "resOccLongestStart": ts(longest["start"]) if longest else "--",
675 "resThrottleMedian": f"{np.median(thr):.1f}" if thr else "0",
676 "resThrottleMax": f"{max(thr):.1f}" if thr else "0",
677 "resSubInBlocks": str(sub_in_blocks) if sub["available"] else "--",
678 "resSubInBlocksShare": pct(sub_share_in_blocks),
679 "resBarrageBlocksN": str(len(bar_blocks)),
680 "resBarrageLongestMin": f"{max(b - a for a, b in bar_blocks) / 60:.1f}" if bar_blocks else "0",
681 "resRecoveryGapLongestMin": f"{(gaps[0][1] - gaps[0][0]) / 60:.1f}" if gaps else "0",
682 "resRecoveryGapLongestStart": ts(gaps[0][0]) if gaps else "--",
683 "resAbsRecoveryN": str(len(abs_windows)),
684 "resAbsRecoveryShare": pct(abs_share),
685 "resAbsGapLongestMin": f"{(abs_gaps[0][1] - abs_gaps[0][0]) / 60:.1f}" if abs_gaps else "0",
686 "resAbsGapLongestStart": ts(abs_gaps[0][0]) if abs_gaps else "--",
687 "resDebtMax": f"{debt.max():.0f}",
688 "resSwellN": str(swell_n),
689 "resSwellFromQuiet": str(swell_from_quiet),
690 "resSwellQuietLevel": f"{quiet_start:.0f}",
691 "resSwellIntoOcc": str(swell_into_occ),
692 "resSwellLarge": str(swell_large),
693 "resOmegaIn": fmt(om["raw_in"], ".2f"),
694 "resOmegaOut": fmt(om["raw_out"], ".2f"),
695 "resOmegaThreeShare": pct(om["raw_three"]),
696 "resOmegaThreeShareIn": pct(om["raw_three_in"]),
697 "resOmegaResIn": fmt(om["res_in"], ".2f"),
698 "resOmegaResOut": fmt(om["res_out"], ".2f"),
699 "resOmegaResThreeShare": pct(om["res_three"]),

```

```

700     "resOmegaResThreeShareIn": pct(om["res_three_in"]),
701     "resOccRows": occ_rows,
702     "resAbsGapRows": abs_rows,
703     "resGapRows": gap_rows,
704 }
705
706 with open(out / "results.tex", "w") as fh:
707     fh.write("% Generated by occupation.py v2. Do not edit by hand.\n")
708     for k, v in macros.items():
709         fh.write(f"\newcommand{{{k}}}{{{v}}}\n")
710
711 print(f"Level: {macros['resLevelMeasure']}")
712 if loud_summary:
713     print(f"Integrated {macros['resIntegrated']} LUFS, LRA {macros['resLRA']} LU")
714 if sub["available"]:
715     print(f"Subtitles: {sub['cues_speech']} speech cues of {sub['cues_total']}, "
716           f"offset {sub['offset_s']:.1f} s, {sub['seconds']} subtitled seconds")
717     print(f"Proxy vs subtitles: precision {macros['resProxyPrecision']}, "
718           f"recall {macros['resProxyRecall']}; D proxy {macros['resDialogueRefProxy']}, "
719           f"D subtitles {macros['resDialogueRefSub']}")
720 print(f"Dialogue reference: {D:.1f} {unit} from {D_source}")
721 print(f"Audible seconds >= reference + {args.delta:.0f} dB: {macros['resExceedTen']}")
722 print(f"Occupation blocks >= {args.min_block:.0f} s: {len(block_rows)}")
723 for r in block_rows:
724     print(f" {ts(r['start'])}-{ts(r['end'])} {r['duration'] / 60:.1f} min "
725           f"p95 {r['p95_rel']:.1f} dB throttle {r['implied_throttle_db']:.1f} dB "
726           f"subtitled s {r['sub_seconds']}")
727 if sub["available"]:
728     print(f"Subtitled speech inside blocks: {sub_in_blocks} s ({macros['resSubInBlocksShare']})")
729 print(f"Longest absolute recovery gap: {macros['resAbsGapLongestMin']} min "
730       f"from {macros['resAbsGapLongestStart']}")
731 print(f"Omega in/out blocks: raw {macros['resOmegaIn']}/{macros['resOmegaOut']}, "
732       f"residualised {macros['resOmegaResIn']}/{macros['resOmegaResOut']}")
733 print("Wrote", out)

```

C.3 Supplementary statistics

```

1  #!/usr/bin/env python3
2  """
3  occupation-extras.py
4  =====
5
6  Supplementary statistics for Chapter 7 and Chapter 8, computed from the
7  per-second table (seconds.csv) and occupation.json that occupation.py
8  writes. Produces results/extras.tex (LaTeX macros) and
9  results/fig-segments.png.
10
11     python3 code/occupation-extras.py --results results
12 """
13
14 import argparse
15 import json
16 from pathlib import Path
17
18 import matplotlib
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 import numpy as np
22 import pandas as pd
23
24 p = argparse.ArgumentParser()
25 p.add_argument("--results", default="results")

```

```

26 p.add_argument("--segment", type=float, default=600.0,
27               help="segment length in seconds for the timeline table")
28 args = p.parse_args()
29
30 R = Path(args.results)
31 s = pd.read_csv(R / "seconds.csv")
32 j = json.loads((R / "occupation.json").read_text())
33 delta = j["delta_db"]
34
35 a = s[s["audible"]].copy()
36 exc = a["rel_db"] >= delta
37 dlg = a[a["dialogue_sub"]] if a["dialogue_sub"].any() else a[a["dialogue_proxy"]]
38 inb = a["in_block"]
39
40
41 def f1(x, spec="+.1f"):
42     return format(float(x), spec) if np.isfinite(x) else "--"
43
44
45 def pc(x):
46     return f"{100 * float(x):.1f}%\\%" if np.isfinite(x) else "--"
47
48
49 def ts(sec):
50     sec = int(round(sec))
51     return f"{sec // 3600:d}:{(sec % 3600) // 60:02d}:{sec % 60:02d}"
52
53
54 m = {}
55 thr = [b["implied_throttle_db"] for b in j["occupation_blocks"]]
56 m["xThrottleMin"] = f1(min(thr), ".1f") if thr else "--"
57
58 # Dialogue spread and dialogue inside blocks
59 q = dlg["rel_db"].quantile([0.1, 0.25, 0.75, 0.9])
60 m["xDlgPTen"], m["xDlgPTwentyFive"] = f1(q[0.1]), f1(q[0.25])
61 m["xDlgPSeventyFive"], m["xDlgPNinety"] = f1(q[0.75]), f1(q[0.9])
62 m["xDlgInBlockMedian"] = f1(dlg.loc[dlg["in_block"], "rel_db"].median())
63 m["xDlgOutBlockMedian"] = f1(dlg.loc[~dlg["in_block"], "rel_db"].median())
64 m["xCenterAdvDlg"] = f1(dlg["center_adv"].median(), ".1f")
65 m["xCenterAdvOther"] = f1(a.loc[~a["dialogue_sub"], "center_adv"].median(), ".1f")
66
67 # Shape of the occupied range
68 o = a.loc[exc, "rel_db"]
69 qo = o.quantile([0.5, 0.9, 0.99])
70 m["xOccMedian"], m["xOccPNinety"], m["xOccPNinetyNine"] = f1(qo[0.5]), f1(qo[0.9]), f1(qo[0.99])
71 m["xOccPlateauShare"] = pc(((o >= 12) & (o < 19)).mean())
72 m["xRelMax"] = f1(a["rel_db"].max())
73 m["xLevelMax"] = f1(a["level_db"].max(), ".1f")
74 m["xOccSeconds"] = str(int(exc.sum()))
75 m["xOccInBlocks"] = str(int((exc & inb).sum()))
76 m["xOccInBlocksShare"] = pc((exc & inb).sum() / max(1, exc.sum()))
77 I = j.get("loudness", {}).get("integrated_lufs", float("nan"))
78 m["xDialogueBelowIntegrated"] = f1(I - j["dialogue_reference"], ".1f")
79
80 # Overdetermination without the exceedance term
81 other_raw = a["omega"] - exc.astype(int)
82 other_res = a["omega_resid"] - exc.astype(int)
83 m["xOtherRawIn"], m["xOtherRawOut"] = f1(other_raw[inb].mean(), ".2f"), f1(other_raw[~inb].mean(), ".2f")
84 m["xOtherResIn"], m["xOtherResOut"] = f1(other_res[inb].mean(), ".2f"), f1(other_res[~inb].mean(), ".2f")
85 for col, key in (("barrage", "Barrage"), ("drum", "Drum"), ("harsh", "Harsh")):
86     m[f"x{key}In"] = f1(a.loc[inb, col].mean(), ".2f")
87     m[f"x{key}Out"] = f1(a.loc[~inb, col].mean(), ".2f")
88
89 # Recovery inside the span of the longest absolute gap and its neighbours
90 gaps = sorted(j["abs_gaps_top5"], key=lambda g: g["start"])
91 # Contiguous cluster containing the longest gap: gaps separated by short windows

```

```

92 longest = max(j["abs_gaps_top5"], key=lambda g: g["end"] - g["start"])
93 cluster = [longest]
94 changed = True
95 while changed:
96     changed = False
97     for g in gaps:
98         if g in cluster:
99             continue
100         lo = min(c["start"] for c in cluster)
101         hi = max(c["end"] for c in cluster)
102         if 0 <= lo - g["end"] <= 30 or 0 <= g["start"] - hi <= 30:
103             cluster.append(g)
104             changed = True
105 c0 = min(c["start"] for c in cluster)
106 c1 = max(c["end"] for c in cluster)
107 span = a[(a["second"] >= c0) & (a["second"] < c1)]
108 m["xSpanStart"], m["xSpanEnd"] = ts(c0), ts(c1)
109 m["xSpanMin"] = f1((c1 - c0) / 60, ".1f")
110 # Qualifying recovery windows: found on the COMPLETE timeline (inaudible seconds included,
111 # no filtering that could make non-consecutive seconds adjacent), then intersected with the span.
112 margin = j.get("recovery_margin_db", 0.0) # defaults for older occupation.json files
113 width = j.get("recovery_width_s", 5.0)
114 calm = ((s["rel_db"] <= margin) | ~s["audible"]).fillna(True).to_numpy()
115 sec = s["second"].to_numpy()
116
117
118 def mask_runs(mask):
119     out, i = [], 0
120     while i < len(mask):
121         if mask[i]:
122             k = i
123             while k + 1 < len(mask) and mask[k + 1]:
124                 k += 1
125             out.append((i, k + 1))
126             i = k + 1
127         else:
128             i += 1
129     return out
130
131
132 windows = [(int(sec[x]), int(sec[y - 1]) + 1) for x, y in mask_runs(calm) if y - x >= width]
133 clipped = [(max(w0, c0), min(w1, c1)) for w0, w1 in windows if w1 > c0 and w0 < c1]
134 span_full = s[(s["second"] >= c0) & (s["second"] < c1)]
135 low_runs = [y - x for x, y in mask_runs(
136     ((span_full["rel_db"] <= margin) | ~span_full["audible"]).to_numpy())]
137 m["xSpanRecoverySeconds"] = str(sum(w1 - w0 for w0, w1 in clipped))
138 m["xSpanRecoveryWindows"] = str(len(clipped))
139 m["xSpanLowSeconds"] = str(sum(low_runs))
140 m["xSpanLowRuns"] = str(len(low_runs))
141 m["xSpanLowLongest"] = str(max(low_runs, default=0))
142 m["xSpanExceedShare"] = pc((span["rel_db"] >= delta).mean())
143 m["xSpanAtOrBelowShare"] = pc((span["rel_db"] <= 0).mean())
144 m["xSpanMedianRel"] = f1(span["rel_db"].median())
145 before = a[a["second"] < c0]
146 m["xBeforeExceedShare"] = pc((before["rel_db"] >= delta).mean())
147 m["xBeforeAtOrBelowShare"] = pc((before["rel_db"] <= 0).mean())
148 m["xBeforeInBlockShare"] = pc(before["in_block"].mean())
149 m["xSpanInBlockShare"] = pc(span["in_block"].mean())
150
151 # Segment table
152 rows = []
153 seg = args.segment
154 for k in range(int(np.ceil(s["second"].max() / seg))):
155     t0, t1 = k * seg, min((k + 1) * seg, s["second"].max() + 1)
156     x = a[(a["second"] >= t0) & (a["second"] < t1)]
157     if len(x) == 0:

```

```

158         continue
159         rows.append((t0, t1, (x["rel_db"] >= delta).mean(), (x["rel_db"] <= 0).mean(),
160                     x["in_block"].mean(), x["dialogue_sub"].mean(), x["rel_db"].median()))
161     m["xSegmentRows"] = " \\\n".join(
162         f"{ts(t0)}--{ts(t1)} & {100 * e:.0f} & {100 * r:.0f} & {100 * b:.0f} & {100 * d:.0f} & {md:+.1f}"
163         for t0, t1, e, r, b, d, md in rows)
164
165     fig, ax = plt.subplots(figsize=(10, 3.2))
166     mid = [(r[0] + r[1]) / 120 for r in rows]
167     ax.plot(mid, [100 * r[2] for r in rows], marker="o", color="tab:red", label=f"at least +{delta:.0f} dB")
168     ax.plot(mid, [100 * r[3] for r in rows], marker="o", color="tab:blue", label="at or below dialogue")
169     ax.plot(mid, [100 * r[5] for r in rows], marker="o", color="0.5", ls="--", label="subtitled speech")
170     ax.set_xlabel("Film time (minutes, segment midpoints)")
171     ax.set_ylabel("Share of audible seconds (%)")
172     ax.set_ylim(0, 100)
173     ax.legend(fontsize=8, loc="upper left")
174     fig.tight_layout()
175     fig.savefig(R / "fig-segments.png", dpi=200)
176     plt.close(fig)
177
178     with open(R / "extras.tex", "w") as fh:
179         fh.write("% Generated by occupation-extras.py. Do not edit by hand.\n")
180         for k, v in m.items():
181             fh.write(f"\newcommand{{\{k\}}}{{\{v\}}}\n")
182
183     for k, v in m.items():
184         if k != "xSegmentRows":
185             print(f"{k:28s} {v}")
186     print(m["xSegmentRows"].replace("\\\\", ""))

```

C.4 Blinded annotation set

```

1  #!/usr/bin/env python3
2  """
3  blind-audition.py
4  =====
5
6  Prepares a blinded annotation set from the audition clips written by
7  analyze-soundtrack-v3.1.
8
9  * copies every detector clip under a random identifier
10 * cuts N random control clips from audible parts of the film
11 * shuffles everything into one folder with one playlist
12 * writes sheet.csv (what the annotator sees) and key.csv (what the
13   annotator must not see until labelling is finished)
14
15 After labelling, join sheet.csv to key.csv on `clip` and pass the
16 result to analyze-soundtrack-v3.1 with --annotations, or analyse it
17 directly. The key has a `category` column in which controls appear as
18 RANDOM_CONTROL.
19 """
20
21 import argparse
22 import random
23 import shutil
24 import subprocess
25 from pathlib import Path
26
27 import numpy as np
28 import pandas as pd
29
30 p = argparse.ArgumentParser()

```

```

31 p.add_argument("input", help="the film (needed to cut control clips)")
32 p.add_argument("--audition", required=True, help="audition/ folder from v3.1")
33 p.add_argument("--frames", required=True, help="frames.csv from v3.1")
34 p.add_argument("--out", default="blind")
35 p.add_argument("--controls", type=int, default=40)
36 p.add_argument("--length", type=float, default=20.0)
37 p.add_argument("--gate-floor", type=float, default=-65.0)
38 p.add_argument("--audio-stream", default="0:a:0")
39 p.add_argument("--seed", type=int, default=None)
40 args = p.parse_args()
41
42 rng = random.Random(args.seed)
43 aud = Path(args.audition)
44 out = Path(args.out)
45 (out / "clips").mkdir(parents=True, exist_ok=True)
46
47 ann = pd.read_csv(aud / "annotations.csv")
48 f = pd.read_csv(args.frames, usecols=["time", "loudness_db"])
49 duration = float(f["time"].max())
50
51 # Control windows: uniformly placed, centre second audible, not
52 # overlapping any detector clip window by more than half its length.
53 f["second"] = np.floor(f["time"]).astype(int)
54 audible = set(f.loc[f["loudness_db"] > args.gate_floor, "second"])
55 taken = [(r["event_seconds"] - r["event_offset_in_clip"],
56          r["event_seconds"] - r["event_offset_in_clip"] + args.length)
57          for _, r in ann.iterrows()]
58
59 controls = []
60 tries = 0
61 while len(controls) < args.controls and tries < 100000:
62     tries += 1
63     a = rng.uniform(0, max(0.0, duration - args.length))
64     if int(a + args.length / 2) not in audible:
65         continue
66     if any(min(b, a + args.length) - max(x, a) > args.length / 2
67            for x, b in taken + controls):
68         continue
69     controls.append((a, a + args.length))
70
71 ids = rng.sample(range(10000, 100000), len(ann) + len(controls))
72 sheet, key = [], []
73
74 for (_, r), cid in zip(ann.iterrows(), ids):
75     name = f"{cid}.mp3"
76     shutil.copyfile(aud / r["clip"], out / "clips" / name)
77     key.append({"clip": name, "category": r["category"], "rank": r["rank"],
78               "event_seconds": r["event_seconds"], "score": r["score"]})
79
80 for (a, b), cid in zip(controls, ids[len(ann):]):
81     name = f"{cid}.mp3"
82     subprocess.run(["ffmpeg", "-y", "-hide_banner", "-loglevel", "error",
83                   "-ss", f"{a:.3f}", "-i", args.input, "-t", f"{b - a:.3f}",
84                   "-map", args.audio_stream, "-vn", "-ac", "2",
85                   "-c:a", "libmp3lame", "-q:a", "4",
86                   str(out / "clips" / name)], check=True)
87     key.append({"clip": name, "category": "RANDOM_CONTROL", "rank": "",
88               "event_seconds": round(a + args.length / 2, 2), "score": ""})
89
90 rng.shuffle(key)
91 for k in key:
92     sheet.append({"clip": k["clip"], "label": "", "role": "", "intensity": "",
93                  "would_reduce": "", "notes": ""})
94
95 pd.DataFrame(sheet).to_csv(out / "sheet.csv", index=False)
96 pd.DataFrame(key).to_csv(out / "key.csv", index=False)

```

```

97 (out / "playlist.m3u").write_text(
98     "\n".join(f"clips/{s['clip']}" for s in sheet) + "\n")
99
100 print(f"{len(ann)} detector clips + {len(controls)} controls -> {out}")
101 print("Give the annotator sheet.csv, playlist.m3u and clips/. Keep key.csv aside.")

```

C.5 Volume behaviour logger for mpv

```

1  -- throttle-log.lua
2  --
3  -- Logs every volume, mute and pause change in mpv against movie time,
4  -- plus a manual marker key, to a CSV beside the video file.
5  --
6  -- Install: copy into mpv's scripts folder
7  -- Windows: %APPDATA%\mpv\scripts\
8  -- Linux: ~/.config/mpv/scripts/
9  --
10 -- During viewing, control volume ONLY in mpv (9 / 0, mouse wheel, m to
11 -- mute). Leave the Windows master volume and the amp at fixed settings.
12 -- Press Shift+A to drop a marker when something bothers you but you
13 -- don't touch the volume.
14 --
15 -- mpv's volume slider is cubic: amplitude = (vol/100)^3, so the gain
16 -- column is 60*log10(vol/100) dB relative to 100%.
17
18 local utils = require "mp.utils"
19
20 local log = nil
21 local started = false
22
23 local function now_pos()
24     return mp.get_property_number("time-pos", -1)
25 end
26
27 local function gain_db(vol)
28     if vol == nil or vol <= 0 then return -120 end
29     return 60 * math.log(vol / 100) / math.log(10)
30 end
31
32 local function write(event, value)
33     if not log then return end
34     local vol = mp.get_property_number("volume", -1)
35     log:write(string.format("%s,%.3f,%.3f,%s,%s,%.1f,%s,%s\n",
36         os.date("!%Y-%m-%dT%H:%M:%SZ"),
37         os.clock(),
38         now_pos(),
39         event,
40         tostring(value),
41         vol,
42         string.format("%.2f", gain_db(vol)),
43         tostring(mp.get_property_bool("mute", false))))
44     log:flush()
45 end
46
47 mp.register_event("file-loaded", function()
48     local path = mp.get_property("path", "")
49     local dir, name = utils.split_path(path)
50     local stamp = os.date("%Y%m%d-%H%M%S")
51     local out = utils.join_path(dir, name .. ".throttle-" .. stamp .. ".csv")
52     log = io.open(out, "w")
53     if not log then
54         mp.osd_message("throttle-log: cannot write " .. out, 5)

```

```
55         return
56     end
57     log:write("utc,clock_s,movie_s,event,value,volume,gain_db,muted\n")
58     started = true
59     write("START", mp.get_property("audio-params/channel-count", "?"))
60     mp.osd_message("throttle-log: recording", 2)
61 end)
62
63 mp.observe_property("volume", "number", function(_, v)
64     if started then write("VOLUME", v) end
65 end)
66
67 mp.observe_property("mute", "bool", function(_, v)
68     if started then write("MUTE", v) end
69 end)
70
71 mp.observe_property("pause", "bool", function(_, v)
72     if started then write("PAUSE", v) end
73 end)
74
75 mp.add_key_binding("A", "throttle-mark", function()
76     write("MARK", "")
77     mp.osd_message("marked", 0.7)
78 end)
79
80 mp.register_event("end-file", function()
81     write("END", "")
82     if log then log:close() end
83     log = nil
84     started = false
85 end)
```

Bibliography

- ANSI (1997). *ANSI S3.5-1997: Methods for Calculation of the Speech Intelligibility Index*. American National Standards Institute, New York.
- ATSC (2013). *A/85: Techniques for Establishing and Maintaining Audio Loudness for Digital Television*. Advanced Television Systems Committee, Washington, DC.
- ATSC (2018). *A/52: Digital Audio Compression (AC-3, E-AC-3) Standard*. Advanced Television Systems Committee, Washington, DC.
- Bello, J. P., Daudet, L., Abdallah, S., Duxbury, C., Davies, M., and Sandler, M. B. (2005). A tutorial on onset detection in music signals. *IEEE Transactions on Speech and Audio Processing*, 13(5), 1035–1047.
- Bregman, A. S. (1990). *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, Cambridge, MA.
- Commercial Advertisement Loudness Mitigation Act (2010). Public Law 111–311, United States.
- Chion, M. (1994). *Audio-Vision: Sound on Screen*. Edited and translated by C. Gorbman. Columbia University Press, New York.
- ComingSoon.net (2026). Spawn’s Michael Jai White gets his Liam Neeson *Taken* moment in *Drive Through Fire* trailer. August 2026. <https://www.comingsoon.net/?p=2181827>.
- EBU (2016). *Tech 3342: Loudness Range, A Measure to Supplement EBU R 128 Loudness Normalisation*. European Broadcasting Union, Geneva.
- EBU (2020). *R 128: Loudness Normalisation and Permitted Maximum Level of Audio Signals*. European Broadcasting Union, Geneva.
- Farina, A. (2000). Simultaneous measurement of impulse response and distortion with a swept-sine technique. *108th Convention of the Audio Engineering Society*, Paris.
- Fastl, H. and Zwicker, E. (2007). *Psychoacoustics: Facts and Models*, 3rd ed. Springer, Berlin.
- Flyxion (2025). Pitch control per speaker (design notes, July 2025, including MuteNet and the whisperizer and mufflerizer proposals). `standardgalactic/kitbash`, file `workspace/Pitch control per speaker.txt`. <https://github.com/standardgalactic/kitbash>.

- Gemmeke, J. F., Ellis, D. P. W., Freedman, D., Jansen, A., Lawrence, W., Moore, R. C., Plakal, M., and Ritter, M. (2017). Audio Set: An ontology and human-labeled dataset for audio events. In *Proceedings of IEEE ICASSP 2017*, 776–780.
- Glass, D. C. and Singer, J. E. (1972). *Urban Stress: Experiments on Noise and Social Stressors*. Academic Press, New York.
- Goodman, S. (2010). *Sonic Warfare: Sound, Affect, and the Ecology of Fear*. MIT Press, Cambridge, MA.
- ITU-R (2015). *Recommendation BS.1770-4: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level*. International Telecommunication Union, Geneva.
- Johnston, J. D. (1988). Transform coding of audio signals using perceptual noise criteria. *IEEE Journal on Selected Areas in Communications*, 6(2), 314–323.
- Kerins, M. (2011). *Beyond Dolby (Stereo): Cinema in the Digital Sound Age*. Indiana University Press, Bloomington.
- Kryter, K. D. (1985). *The Effects of Noise on Man*, 2nd ed. Academic Press, Orlando.
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., and Nieto, O. (2015). librosa: Audio and music signal analysis in Python. In *Proceedings of the 14th Python in Science Conference*, 18–25.
- Miedema, H. M. E. and Oudshoorn, C. G. M. (2001). Annoyance from transportation noise: Relationships with exposure metrics DNL and DENL and their confidence intervals. *Environmental Health Perspectives*, 109(4), 409–416.
- Moore, B. C. J. (2012). *An Introduction to the Psychology of Hearing*, 6th ed. Emerald, Bingley.
- Murch, W. (2005). Dense clarity, clear density. *The Transom Review*, 5(1).
- Schafer, R. M. (1977). *The Tuning of the World*. Knopf, New York.
- Sergi, G. (2004). *The Dolby Era: Film Sound in Contemporary Hollywood*. Manchester University Press, Manchester.
- SMPTE (2012). *RP 200: Relative and Absolute Sound Pressure Levels for Motion-Picture Multichannel Sound Systems*. Society of Motion Picture and Television Engineers, White Plains, NY.
- Vickers, E. (2010). The loudness war: Background, speculation, and recommendations. *129th Convention of the Audio Engineering Society*, San Francisco.